Artefact 2 — AI interaction log (Claude, Anthropic)

Model: Claude Opus 5.5 (claude-opus-5-5), used via Claude Cowork. Dates: October 29– November 2026. Prompts are reproduced in full. AI responses are reproduced in full and unedited. Tool actions the AI took (running code, reading or writing files) are listed as one-line entries; their raw outputs are omitted. Attached screenshots are noted as [image]. Note: this log begins when the session's context was compacted on 3 November 2026. Earlier Artefact 2 exchanges (from 1–3 November) are summarised in the first entry and are not reproduced verbatim here.

Prompt 1 (2026-10-29 04:45 UTC)

if we merge those two projects together, is it possible to have data.ingestion.py do everything instead of having to route the second half with another command

Response 1

Yes. Either file could run the whole thing, so the user types one command. But I'd put the orchestration in `prolog_interface.py` rather than `data_ingestion.py`.

How it would work from `data_ingestion.py`

You'd add one final stage after A1's existing steps (fetch → validate → reproject → join → write `intersection_sites.csv`):

1. Write `intersection_sites.csv` straight into `hybrid_ai_starter/processed/`. That removes the copy step.
2. Pass on a flag such as `--site-key` or `--all`, and launch `prolog_interface.py` with a subprocess call. That then trains and compares the models, picks the best one, predicts and runs the Prolog rules.
3. Run A2 only if A1's validation passed. If pandera quarantines rows or the data contract fails, the run stops before any model trains on bad data.

```
python3 data_ingestion.py --all
```

Why the other direction is better

```
python3 prolog_interface.py --from-a1 "<A1 folder>/Source" --all
```

Here, `prolog_interface.py` runs `data_ingestion.py` first, checks that its output CSV has the columns A2's models expect, uses that CSV, then carries on as it does now. It's still one command, but:

- **A1 is already submitted and marked.** If you change `data_ingestion.py`, the A1 code in your merged project no longer matches what you handed in. Keeping A1 untouched means you can say "A1 runs exactly as submitted, and A2 builds on it."
- **Integration is your job in A2.** `prolog_interface.py` is your file, and it's already described as the thing that connects everything. Putting the A1 hook there makes your contribution clearer, and your teammates' files stay unchanged.
- **The direction of dependency makes sense.** A2 needs A1's data, but A1 doesn't need A2. A1 stays a standalone data pipeline anyone can reuse.

Things to handle either way

- **Dependencies:** the two `requirements.txt` files need merging into one environment. I'd check that the pandas and scikit-learn versions don't clash.

- **Run time:** A1 downloads from the CKAN API on every run. I'd add an option to skip the download and reuse the last verified files (A1's SHA-256 manifest already tells you whether they're unchanged), so a demo doesn't depend on the network.
- **Schema check:** if a fresh download changes columns or types, the run should fail with a clear message rather than a confusing model error halfway through.
- **The interactive menu:** if you run it with no flag, A2 still asks you to pick a site. That's fine for a live demo, but `--all` or `--site-key` keeps it fully automatic.

Do you want me to build the `--from-a1` version into `prolog_interface.py`? I'd need the A1 `Source` folder connected again (or attached), since my copy of A1's outputs is gone.

Prompt 2 (2026-10-29 04:57 UTC)

lets take a step back, does your revised prolog point to the best model if it exists or does it retrain the data and over rides the current best model with a new one

Response 2

It always retrains, and the new model replaces the old `best_model.joblib`. It never reuses an existing one. That's the design you asked for, so the examiner sees the comparison happen on every run.

Each time you run `prolog_interface.py`, `train_and_select_model()` does this:

1. **Deletes the old `models/best_model.joblib`** if it's there. That way a file left over from an earlier run can't be picked up by mistake.
2. **Runs `train_models.py`**, which trains and compares all the models and saves the winner as a new `best_model.joblib`.
3. **Checks that the file was actually created.** If it wasn't, the run stops with an error instead of carrying on with nothing.
4. **Loads that new file** and prints which model it is (for example `RandomForestClassifier`), then goes on to prediction and Prolog.

So yes, every run overwrites the best model.

Overwriting doesn't make results drift. Training uses a fixed seed (42), the same 75/25 split and the same CSV, so rerunning gives you the same winning model and the same predictions every time. The result changes only if the input data changes, for example after a fresh A1 download.

The `train_models.py` currently in your folder is the older two-model version, and it doesn't save `best_model.joblib`. With that version, step 3 stops the run with the message that it "did not save best_model.joblib". It will work once your teammate's four-model version (which saves `best_model.joblib`) is in the folder.

Prompt 3 (2026-10-29 05:00 UTC)

if my teammate revises his train_model to retrain the data and to create a new best model, would that fit seamlessly into the thing prolog interface uve created?,

Response 3

Yes, it should drop straight in. `prolog_interface.py` was built expecting exactly that. It works as long as the revised `train_models.py` meets these conditions:

1. **It saves the winner to exactly `models/best_model.joblib`.** The interface looks for that path and nothing else, and it stops with an error if the file isn't created.
2. **It runs without asking for input.** The interface runs it with no keyboard input, so an `input()` prompt would fail or hang.
3. **It exits normally when it succeeds.** If it crashes or ends with an error code, the interface stops and shows its output.
4. **The saved file is the whole pipeline, not just the classifier.** That means the imputers, scaler and one-hot encoder plus the model, saved as one sklearn `Pipeline`. The prediction step passes in raw site features, so the preprocessing has to be inside the saved file. This matters most for XGBoost, which is easy to save on its own by accident.
5. **The feature columns match `predict_site.py`.** Prediction builds the input from its own list of 15 numeric and 2 categorical features. If your teammate adds, removes or renames any, `predict_site.py` needs the same change, or prediction will fail.
6. **It predicts the same target the same way.** The target is `any_fsi`, and class 1 has to mean "FSI present", because the Prolog facts are built from the class-1 probability.
7. **Its packages are in `requirements.txt`**, including `xgboost`.
8. **It finishes within 10 minutes.** That's the time limit the interface allows. Four models with 5-fold cross-validation on 677 rows should take seconds.

If he meets all eight, you don't need to change anything in `prolog_interface.py`. It runs his script, picks up the new `best_model.joblib`, prints which model won, and carries on to prediction and Prolog.

When he sends the file, I can check it against this list and do a full test run.

Prompt 4 (2026-10-29 05:03 UTC)

[image]

we were talking about merging the two artefacts with the prolog interface option and he said this, im not sure what he means

Response 4

Kieren is saying his pull request (PR) already does what you need, but it isn't in the main code yet.

What each line means

- **"point it at best_model.joblib and it'll work":** his version of `train_models.py` saves the winning model as `best_model.joblib`. Your interface just needs to load that file. Yours already does. It loads `models/best_model.joblib` after running his script, so you don't need to change anything.
- **"when my pr is merged":** his changes are on a separate branch on GitHub. Until someone merges them into the main branch, the `train_models.py` everyone downloads is still the old two-model version. That explains why your copy doesn't save `best_model.joblib` and your interface currently stops at that check. Once it's merged and you pull the latest code, the error goes away.
- **"same directory":** `best_model.joblib` is saved in the same `models/` folder as the other model files, which is where your interface looks.

On retraining: he didn't answer that directly, but it comes for free. Running `train_models.py` always trains every model from scratch on the CSV and writes a new `best_model.joblib`. Your interface runs it on every launch, so each run retrains and replaces the best model. His script doesn't need anything extra for that.

What he hasn't confirmed are the two points most likely to break: whether the saved file includes the preprocessing (the whole pipeline), and whether his feature list still matches `predict_site.py`. You could send him something like:

> Sweet. Two quick checks so it plugs straight in: is best_model.joblib saved as the full pipeline (preprocessing + model), and is the feature list the same as in predict_site.py? Also is xgboost in requirements.txt?

If you can open the PR on GitHub, the "Files changed" tab shows his new `train_models.py`. Paste it here, or connect the folder after it's merged, and I'll check it against the earlier list and do a full test run before you merge A1 and A2.

Prompt 5 (2026-10-29 05:04 UTC)

[image]

Response 5

That's a good sign. Kieren's PR updates `predict_site.py` so it loads `models/best_model.joblib` instead of `models/logistic_regression.joblib`. It was one of the fixes I'd flagged for your teammates.

It works with your interface without changes. Your `prolog_interface.py` already sets `predict_site.MODEL_FILE` to that same path after training, so both now point at the same file. Once the PR is merged, that override does nothing new, but it causes no conflict. It also means `predict_site.py` works correctly when someone runs it on its own, not only through your interface.

What this diff doesn't show:

- **The `train_models.py` changes.** This is the part that actually creates `best_model.joblib`, so it's the one to check. Scroll to that file in the "Files changed" tab.
- **Whether the "Logistic Regression" label was updated.** `predict_site.py` used to write `"Logistic Regression"` as the model name in `ml_prediction.json`. If that line hasn't changed, running `predict_site.py` on its own would mislabel a Random Forest winner. Your interface already writes the real model name, so this only matters outside it. If you search the PR for "Logistic Regression", you'll see whether it's still there.

If you screenshot the `train_models.py` diff, mainly the save step and the feature lists, I can confirm the rest.

Prompt 6 (2026-10-29 05:06 UTC)

[image]

Response 6

This screenshot is the PR's `.gitignore`, not `train_models.py`. A `.gitignore` lists files Git shouldn't upload, usually generated outputs and local clutter. It doesn't affect how anything runs, but it shows how Kieren has laid out the repo.

What it tells us

- **The repo already holds both artefacts.** `crash-risk-ingestion/` (your A1 pipeline) now sits next to `hybrid_ai_starter/` (A2). That makes merging the two easier: `prolog_interface.py` can find A1's output by a fixed relative path, `../crash-risk-ingestion/data/processed/intersection_sites.csv`, with no folder argument needed.
- **A1's downloaded data and outputs aren't uploaded (lines 16–20).** Anyone who clones the repo must run `data_ingestion.py` online to fetch the data. The comment says `--offline` won't work because `Data/`

doesn't include the crash archive. For a demo, that means you need internet access, or a local copy of the data already in place.

- **`lachinemearning/` (line 14)** is a local copy of your original A1 repo. It's ignored so it isn't uploaded twice.
- **A2's per-run outputs aren't uploaded (lines 22–28),** but the model metrics in `output/` are. That's sensible: the comparison results stay visible, and the per-site files are regenerated each run.

Two gaps relevant to your interface

1. **Your `--all` outputs aren't on the list:** `hybrid_results_all.csv`, `hybrid_summary.json`, `all_site_facts.pl`, `all_ml_prediction_facts.pl` and `pipeline.log`. If the list stops at line 28, those would get uploaded whenever someone runs `--all`. If you want to keep them out, add them, or simply ignore `hybrid_ai_starter/output/*` except the metrics files.
2. **I can't see whether `models/*.joblib` is ignored.** If `best_model.joblib` is uploaded, the repo would contain a pre-made model, which is the "old output" look you wanted to avoid. Your interface deletes and recreates it on every run anyway, but ignoring `hybrid_ai_starter/models/*.joblib` keeps the repo clean.

The `train_models.py` diff is still the one I need, mainly the line that saves `best_model.joblib` and the feature lists near the top.

Prompt 7 (2026-10-29 05:08 UTC)

back it up, does kieren need a pr for the models_train because i cant find it in any pr requests

Response 7

Only if `train_models.py` on the main branch doesn't already save `best_model.joblib`. If you can't find it in any PR, one of three things is true:

1. **It's already on main.** Someone pushed it straight to the main branch without a PR. Then nothing else is needed, and you just pull the latest code.
2. **It's on a branch with no PR yet.** The work exists but hasn't been proposed for merging.
3. **It hasn't been done.** Nobody has changed `train_models.py` yet.

How to check (about a minute on GitHub)

- Open `hybrid_ai_starter/train_models.py` on the **main** branch and press Ctrl+F (Cmd+F on Mac) for `best_model`. If you find a line like `joblib.dump(..., "best_model.joblib")`, it's case 1, and you're covered.
- If it isn't there, click **History** on that file to see the latest commits. Then check the branch dropdown for other branches with newer work.

Why this matters: the copy of `train_models.py` I tested (from the repo you connected earlier) only trains LR and DT and saves `logistic_regression.joblib` and `decision_tree.joblib`, not `best_model.joblib`. If main still looks like that, Kieren's PR only changes the **reader** (`predict_site.py` now loads `best_model.joblib`), and nothing **writes** that file. Both `predict_site.py` and your interface would then stop with "best_model.joblib not found".

You also mentioned a teammate saying the four-model comparison (DT, LR, XGBoost, RF) already exists. If that's true, it's likely on main already or on someone's branch. It may not be Kieren's work either, since `train_models.py` was originally written by a different teammate (dobi0016). It's worth asking in the chat who has the four-model `train_models.py` and whether it's on main.

Prompt 8 (2026-10-29 05:08 UTC)

document: “““ File: train_models.py Student: dobi0016 Course: COMP3742/9742 - Artificial Intelligence

Purpose: Train and compare Logistic Regression and Decision Tree classifiers for the intersection crash-history target any_fsi.

Important: Crash counts, injury statistics, crash rates, and fields derived from crash outcomes are deliberately excluded from the predictors. This is an educational prototype, not a validated road-safety tool. “““

import json from pathlib import Path

import joblib import pandas as pd

from sklearn.compose import ColumnTransformer from sklearn.impute import SimpleImputer from sklearn.linear_model import LogisticRegression from sklearn.metrics import (accuracy_score, average_precision_score, classification_report, confusion_matrix, f1_score, precision_score, recall_score, roc_auc_score,) from sklearn.model_selection import (cross_val_score, train_test_split,) from sklearn.pipeline import Pipeline from sklearn.preprocessing import OneHotEncoder, StandardScaler from sklearn.tree import DecisionTreeClassifier

---

1. File locations and settings

BASE_DIR = Path(**file**).resolve().parent

DATA_FILE = BASE_DIR / “processed” / “intersection_sites.csv” MODEL_DIR = BASE_DIR / “models” OUTPUT_DIR = BASE_DIR / “output”

TARGET = “any_fsi” RANDOM_STATE = 42 TEST_SIZE = 0.25 CV_FOLDS = 5

---

2. Select leakage-conscious predictor features

These features describe the site and its context.

Crash outcomes, injury counts, crash rates, and other

outcome-derived columns are intentionally not included.

NUMERIC_FEATURES = [“n_points”, “point_spread_m”, “spread_flag”, “entering_aadt”, “major_road_aadt”, “n_state_roads”, “cv_percent”, “volume_stale”, “has_volume”, “ped_crossings_200m”, “school_crossings_300m”, “dist_school_crossing_m”, “dist_cbd_km”, “lon”, “lat”,]

CATEGORICAL_FEATURES = [“site_type”, “road_no”,]

FEATURES = NUMERIC_FEATURES + CATEGORICAL_FEATURES

---

3. Load and validate the dataset

---

def load_dataset(): “““Load the CSV and check that the required columns exist.”””

```
if not DATA_FILE.is_file():
    raise FileNotFoundError(
        f"Dataset not found: {DATA_FILE}\n"
        "Check that intersection_sites.csv is in the processed folder."
    )

data = pd.read_csv(DATA_FILE)

required_columns = FEATURES + [TARGET]
missing_columns = [
    column for column in required_columns
    if column not in data.columns
]

if missing_columns:
    raise ValueError(
```

```
        "The dataset is missing required columns: "
        + ", ".join(missing_columns)
    )

# Only rows with a known target can be used for supervised training.
data[TARGET] = pd.to_numeric(data[TARGET], errors="coerce")
data = data.dropna(subset=[TARGET]).copy()

# The target must contain binary 0/1 values.
invalid_targets = ~data[TARGET].isin([0, 1])

if invalid_targets.any():
    raise ValueError(
        "The target column must contain only 0, 1, or missing values."
    )

data[TARGET] = data[TARGET].astype(int)

if data[TARGET].nunique() != 2:
    raise ValueError(
        "Training requires examples of both target classes."
    )

if data[TARGET].value_counts().min() < CV_FOLDS:
    raise ValueError(
        f"Each target class needs at least {CV_FOLDS} examples "
        "for the selected cross-validation."
    )

return data
```

---

4. Build preprocessing and model pipelines

---

def build_pipeline(model): “””Create a pipeline that handles missing and categorical values.”””

```
numeric_pipeline = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ]
)

categorical_pipeline = Pipeline(
    steps=[
        (
            "imputer",
            SimpleImputer(strategy="most_frequent"),
```

```
        ),
        (
            "encoder",
            OneHotEncoder(handle_unknown="ignore"),
        ),
    ]
)

preprocessor = ColumnTransformer(
    transformers=[
        ("numeric", numeric_pipeline, NUMERIC_FEATURES),
        (
            "categorical",
            categorical_pipeline,
            CATEGORICAL_FEATURES,
        ),
    ]
)

return Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("classifier", model),
    ]
)
```

---

5. Evaluate one model

---

def evaluate_model(name, model, X_train, X_test, y_train, y_test): “““Train a model, evaluate it, and save its files.”“”

```
print(f"\nTraining {name}...")

model.fit(X_train, y_train)

predictions = model.predict(X_test)
probabilities = model.predict_proba(X_test)[:, 1]

# Cross-validation uses the training data only.
cv_scores = cross_val_score(
    model,
    X_train,
    y_train,
    cv=CV_FOLDS,
    scoring="f1",
    n_jobs=-1,
)
```

```
metrics = {
    "model": name,
    "target": TARGET,
    "features": FEATURES,
    "training_rows": int(len(X_train)),
    "test_rows": int(len(X_test)),
    "test_accuracy": float(
        accuracy_score(y_test, predictions)
    ),
    "test_precision": float(
        precision_score(
            y_test,
            predictions,
            zero_division=0,
        )
    ),
    "test_recall": float(
        recall_score(
            y_test,
            predictions,
            zero_division=0,
        )
    ),
    "test_f1": float(
        f1_score(
            y_test,
            predictions,
            zero_division=0,
        )
    ),
    "test_roc_auc": float(
        roc_auc_score(y_test, probabilities)
    ),
    "test_average_precision": float(
        average_precision_score(y_test, probabilities)
    ),
    "cv_f1_mean": float(cv_scores.mean()),
    "cv_f1_std": float(cv_scores.std()),
    "confusion_matrix": confusion_matrix(
        y_test,
        predictions,
        labels=[0, 1],
    ).tolist(),
    "classification_report": classification_report(
        y_test,
        predictions,
        labels=[0, 1],
        target_names=["no_fsi", "any_fsi"],
        output_dict=True,
        zero_division=0,
    ),
```

```
}

# Save the trained pipeline.
model_path = MODEL_DIR / f"{name}.joblib"
joblib.dump(model, model_path)

# Save a readable record of the evaluation.
metrics_path = OUTPUT_DIR / f"{name}_metrics.json"
metrics_path.write_text(
    json.dumps(metrics, indent=2) + "\n",
    encoding="utf-8",
)

print(f"Test accuracy: {metrics['test_accuracy']:.3f}")
print(f"Test precision: {metrics['test_precision']:.3f}")
print(f"Test recall: {metrics['test_recall']:.3f}")
print(f"Test F1-score: {metrics['test_f1']:.3f}")
print(f"Test ROC-AUC: {metrics['test_roc_auc']:.3f}")
print(
    "Test average precision: "
    f"{metrics['test_average_precision']:.3f}"
)
print(
    f"{CV_FOLDS}-fold CV F1: "
    f"{metrics['cv_f1_mean']:.3f} "
    f"+/- {metrics['cv_f1_std']:.3f}"
)
print("Confusion matrix:", metrics["confusion_matrix"])
print("Saved model:", model_path)
print("Saved metrics:", metrics_path)

return metrics
```

---

6. Train and compare both classifiers

---

def main(): “””Run the complete model training and comparison process.”””

```
MODEL_DIR.mkdir(parents=True, exist_ok=True)
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

data = load_dataset()

X = data[FEATURES].copy()
y = data[TARGET].copy()

print("Intersection crash-history classification")
```

```
print("----------------------------------------")
print("Rows with known target:", len(data))
print("Target counts:")
print(y.value_counts().sort_index().to_string())
print("\nPredictor features:")
for feature in FEATURES:
    print("-", feature)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=TEST_SIZE,
    random_state=RANDOM_STATE,
    stratify=y,
)

models = {
    "logistic_regression": build_pipeline(
        LogisticRegression(
            max_iter=2000,
            class_weight="balanced",
            random_state=RANDOM_STATE,
        )
    ),
    "decision_tree": build_pipeline(
        DecisionTreeClassifier(
            max_depth=5,
            min_samples_leaf=5,
            class_weight="balanced",
            random_state=RANDOM_STATE,
        )
    ),
}

all_metrics = {}

for name, model in models.items():
    all_metrics[name] = evaluate_model(
        name,
        model,
        X_train,
        X_test,
        y_train,
        y_test,
    )

comparison = []

for name, metrics in all_metrics.items():
    comparison.append(
        {
            "model": name,
            "test_accuracy": metrics["test_accuracy"],
```

```
            "test_precision": metrics["test_precision"],
            "test_recall": metrics["test_recall"],
            "test_f1": metrics["test_f1"],
            "test_roc_auc": metrics["test_roc_auc"],
            "test_average_precision": (
                metrics["test_average_precision"]
            ),
            "cv_f1_mean": metrics["cv_f1_mean"],
            "cv_f1_std": metrics["cv_f1_std"],
        }
    )

comparison_path = OUTPUT_DIR / "model_comparison.json"
comparison_path.write_text(
    json.dumps(comparison, indent=2) + "\n",
    encoding="utf-8",
)

print("\nModel comparison")
print("----------------")
print(
    pd.DataFrame(comparison)
    .round(3)
    .to_string(index=False)
)
print("\nComparison saved to:", comparison_path)
print("\nTraining and evaluation complete.")
```

if **name** == "**main**": main()

Response 8

This is the old two-model version, the same one I tested earlier. It trains Logistic Regression and Decision Tree only, and saves `logistic_regression.joblib` and `decision_tree.joblib`. Nothing in it writes `best_model.joblib`.

So yes, `train_models.py` needs a change, and it isn't in any PR. That also means Kieren's PR on its own would break things. Once it's merged, `predict_site.py` loads `best_model.joblib`, but no script creates that file, so both `predict_site.py` and your interface would stop with "file not found". The four-model comparison you heard about isn't on main either, at least not in this file.

This is dobi0016's file, so it's worth agreeing in the team chat who makes the change. The change is small:

1. Save the best model (required). Add this at the end of `main()`, after the comparison is printed:

```
    best = max(comparison, key=lambda row: row["cv_f1_mean"])
    best_name = best["model"]
    best_path = MODEL_DIR / "best_model.joblib"
    joblib.dump(models[best_name], best_path)

    (OUTPUT_DIR / "best_model.json").write_text(
        json.dumps(
            {
                "best_model": best_name,
                "selection_metric": "cv_f1_mean",
                "score": best["cv_f1_mean"],
```

```
            },
            indent=2,
        ) + "\n",
        encoding="utf-8",
    )

    print(f"\nBest model: {best_name} (CV F1 {best['cv_f1_mean']:.3f})")
    print("Saved best model:", best_path)
```

It picks the winner by cross-validated F1, which is calculated on the training data only. That's a fairer choice than test-set scores: if you pick the winner by test score, that score is no longer an honest measure of how well it performs. `models[best_name]` is already trained at this point, and the whole pipeline (preprocessing plus model) is saved, which is what your interface needs.

2. Add Random Forest and XGBoost (for the four-model comparison). Add these imports:

```
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
```

and add these entries to the `models` dictionary:

```
        "random_forest": build_pipeline(
            RandomForestClassifier(
                n_estimators=300,
                min_samples_leaf=5,
                class_weight="balanced",
                random_state=RANDOM_STATE,
                n_jobs=-1,
            )
        ),
        "xgboost": build_pipeline(
            XGBClassifier(
                n_estimators=300,
                max_depth=4,
                learning_rate=0.05,
                eval_metric="logloss",
                random_state=RANDOM_STATE,
            )
        ),
```

Then add `xgboost` to `requirements.txt`. The docstring and the section-6 heading also need updating, since they say "both classifiers".

Checked against the earlier list, with these changes:

- It saves to `models/best_model.joblib`.
- It runs with no input prompts.
- The saved file is the full pipeline.
- The features are unchanged, so they still match `predict_site.py`.
- The target is `any_fsi`, with class 1 meaning FSI present.
- It should run in well under a minute.

So it plugs into your interface as is. Merge Kieren's PR and this change together, so the reader and the writer of `best_model.joblib` arrive at the same time.

Prompt 9 (2026-10-30 05:10 UTC)

if we take a step back and look at the transition between kierens code and mine, if what he says is true in the screenshot where are we at

Response 9

If Kieren is right that merging his PR makes `best_model.joblib` appear in `models/`, you're almost done. Your side needs no code changes.

Where each piece stands (assuming he's right)

Piece	Owner	Status
`train_models.py` trains and compares the models, then writes `models/best_model.joblib`	Kieren's PR (per his message)	Not yet seen. It isn't in the version on main.
`predict_site.py` reads `models/best_model.joblib`	Kieren's PR	Done (the diff you showed)
`prolog_interface.py` runs training, checks `best_model.joblib` was created, predicts, runs Prolog	You	Done and tested. No change needed.
A1 and A2 in one repo	Kieren's PR (`crash-risk-ingestion/` next to `hybrid_ai_starter/`)	Folder layout done. The single-command link isn't built yet.

How the handoff works once merged

`prolog_interface.py` → runs his `train_models.py` → it compares the models and writes `best_model.joblib` → your interface loads that file (and so does `predict_site.py`, since both point at the same path) → prediction → Prolog → result.

His code and yours meet at one point: the file `models/best_model.joblib`. He writes it, and you read it.

What's still unconfirmed

The writer. On main, `train_models.py` doesn't save `best_model.joblib`. So for his claim to hold, his PR must change `train_models.py` too. Check that PR's "Files changed" list. If `train_models.py` isn't in it, ask him:

> Which file in your PR writes best_model.joblib? The train_models.py on main only saves logistic_regression and decision_tree.

If it isn't there, the code I gave in my last message fills that gap.

Next steps, in order

1. Confirm that the writer is in his PR, or add it.
2. Merge the PR and pull the latest code.
3. Do one test run: `python3 prolog_interface.py --all`. If you connect the updated folder, I can do this for you.
4. Then connect the two artefacts. Since A1 now lives in `crash-risk-ingestion/` in the same repo, `prolog_interface.py` can run `data_ingestion.py` and read its output by a fixed relative path. That gives you one command from A1 to A2.

Prompt 10 (2026-10-30 05:12 UTC)

what do you mean by prediction -> prolog -> results

Response 10

Those are the last three stages inside `prolog_interface.py`, after training has produced `best_model.joblib`. Here they are for one site, using the example from our earlier test run (Main North Rd / Regency Rd).

1. Prediction (Python, ML)

- `prepare_site.py` pulls that site's row from `intersection_sites.csv`: traffic volume, nearby pedestrian and school crossings, distance to the CBD, and so on.
- The best model takes those features and outputs a probability that the site has a history of fatal or serious-injury crashes (`any_fsi`). Here that was **0.754**, which is above 0.5, so the classification is "FSI present".
- That result is written as **Prolog facts**, which are plain-text statements Prolog can read:

```
ml_classification(site_id, historical_fsi_present).
ml_positive_probability(site_id, 0.754).
```

- The site's own data is written as facts too, such as `traffic_volume(site_id, ...)`, `pedestrian_crossings(site_id, ...)` and `volume_current(site_id)`.

2. Prolog (symbolic rules)

- The interface starts SWI-Prolog and loads `crash_risk_rules.pl` along with those fact files.
- The rules combine the ML output with the site facts. They are checked in this order:
  1. **Data issues** (missing or stale traffic volume, missing ML output) → `data_review`. You can't trust a judgement built on bad data.
  2. ML says "FSI present" **and** there's a context concern (for example traffic above 20,000, or many crossings nearby) → `priority_safety_review`
  3. ML says "FSI present", with no context concern → `safety_review`
  4. ML says "absent", but there is a context concern → `contextual_review`
  5. Otherwise → `routine_monitoring`
- For this site, the ML result was "present" and traffic was above the threshold, so the result is **`priority_safety_review`**, with the reason `higher_traffic_exposure`.

3. Results (back in Python)

- Prolog prints one tagged line per site. The interface reads it and turns it into:
  - an on-screen summary: site, model used, probability, action, reason and any data issues
  - `output/hybrid_result.json` for a single site, or `hybrid_results_all.csv` plus `hybrid_summary.json` with `--all`

Why it's split this way: the ML model finds a statistical pattern ("sites like this tend to have FSI crashes"), but it can't explain itself. The Prolog rules add clear reasoning on top: they check the data is trustworthy, add context, and turn a probability into a recommended action with a stated reason. That combination is what makes the system "hybrid", which is the point of Artefact 2.

Prompt 11 (2026-10-30 05:13 UTC)

"my prolog_interface runs ur train_models.py if ur train_models.py compares the models and writes the best model.joblib my interfact loads the file and then does the predictions (prepare_site.py) and then the prolog stuff" is this correct

Response 11

Almost. The only fix is which script does what: `prepare_site.py` gets the site data ready, and `predict_site.py` makes the prediction. Prepare comes before predict.

The corrected version:

> My `prolog_interface.py` runs your `train_models.py`. If it compares the models and writes `best_model.joblib`, my interface loads that file. Then it uses `prepare_site.py` to pull the site's data and write it as Prolog facts, and `predict_site.py` to run the best model and write the prediction as Prolog facts. Then it runs the Prolog rules on those facts and outputs the result.

The order is:

1. **Train**: `train_models.py` (Kieren's or dobi0016's) writes `best_model.joblib`
2. **Prepare**: `prepare_site.py` loads the site's row and writes the site facts
3. **Predict**: `predict_site.py` uses `best_model.joblib` to get a probability and writes the ML facts
4. **Reason**: Prolog runs `crash_risk_rules.pl` on both sets of facts and decides the action and reason
5. **Result**: your interface prints it and saves the JSON/CSV

Your interface runs all five steps. The teammates' files do the individual jobs.

Prompt 12 (2026-10-30 05:16 UTC)

does that align with what kieren said about just point it to best_model?

Response 12

Yes, it fits with what he said. Your interface just adds one step before his.

What Kieren means: load `models/best_model.joblib` instead of `logistic_regression.joblib`. Your interface does that. After training, it loads exactly that file, and `predict_site.py` (with his PR) reads the same path. The two of you agree on the handoff point.

What your interface adds: it also runs `train_models.py` first, so `best_model.joblib` is recreated on every run. Kieren's message reads as if he expects the file to already exist, from someone running `train_models.py` once beforehand, as in the README steps. That isn't a conflict. Both approaches end with "use `best_model.joblib`". Yours just guarantees the file is fresh and shows the model comparison happening, which was your reason for doing it.

	Kieren's view	Your interface
Who creates `best_model.joblib`	someone runs `train_models.py` by hand first	the interface runs it automatically
Which model gets used	`best_model.joblib`	`best_model.joblib`
What happens on rerun	reuses the existing file	retrains, replacing the old file (same result with the same data)

The only thing it depends on is the same as before: his `train_models.py` must actually write `best_model.joblib`. If it does, "point it at best_model" and your design work together without changes.

Prompt 13 (2026-10-30 05:18 UTC)

okay so thats the artefact 2 part done, how can we tie the knot between artefact 1 and 2 starting from the prolog_interface.py and calling back to artefact 1 like we mentioned before

Response 13

[Tool action: device_list_dir — /Users/aftabhussaini/Downloads/HybridAI-main]

[Tool action: Bash — Inspect A1 source in workspace]

[Tool action: Bash — Locate A1 pipeline files]

[Tool action: Bash — Unpack A1 package to inspect]

[Tool action: Bash — Read A1 entry point and config]

[Tool action: Bash — Locate current interface and structure]

[Tool action: Bash — Read header, constants, arg parsing and main]

[Tool action: Bash — Read training stage function]

[Tool action: Bash — Add Artefact 1 ingestion stage to interface]

[Tool action: Bash — View section headers around insertion]

[Tool action: Bash — Fix section order and check arg validation]

[Tool action: Bash — Build merged repo layout for testing]

[Tool action: Bash — Install A1 deps and check data portal reachability]

I can't reach the SA data portal from this workspace, so I'll test the handoff with a stand-in A1 script that writes the real A1 output CSV, and test the failure paths too.

[Tool action: Bash — Test A1-to-A2 handoff and failure paths]

All paths work. Saving it to your folder:

[Tool action: Bash — Stage updated interface for delivery]

[Tool action: device_list_dir — /Users/aftabhussaini/Downloads/HybridAI-main/hybrid_ai_starter]

[Tool action: device_stage_files — ['/Users/aftabhussaini/Downloads/HybridAI-main/hybrid_ai_starter/prolog_interface.py']]

[Tool action: Bash — Compare device copy with updated version]

[Tool action: device_commit_files —]

`prolog_interface.py` can now run Artefact 1 first, so the whole system from A1 to A2 is one command. It's saved in your `hybrid_ai_starter` folder.

```
cd hybrid_ai_starter
python3 prolog_interface.py --from-a1 --all
python3 prolog_interface.py --from-a1 --site-key signal:TS:025
python3 prolog_interface.py --from-a1 --a1-offline --all
```

`--a1-offline` reuses A1's cached downloads instead of fetching again, which is safer for a live demo. Without `--from-a1`, the interface behaves exactly as before, so A2 still runs on its own.

What `--from-a1` adds before training

1. It runs `data_ingestion.py` in `../crash-risk-ingestion/`, which is where Kieren's PR puts it. You can point it somewhere else with `--a1-dir PATH`. A1's own progress prints live.

2. If A1 fails, it stops with "Artefact 2 was not run", so no model ever trains on half-processed data.
3. It checks that A1's `intersection_sites.csv` has every column A2 needs. It takes that list from the teammates' own `train_models.py` and `prepare_site.py`, so the check stays in step if they change their features.
4. It saves the current A2 CSV as `intersection_sites.previous.csv`, then copies A1's output into `processed/`.
5. It prints A1's run report: raw, clean and rejected rows per source layer. That's good evidence for the examiner that the validation ran.

After that it carries on with the usual train → prepare → predict → Prolog → result.

Tested here:

- The full run, both `--all` and a single site. The single site gave the same result as before (Main North/Regency, 75.4% → `priority_safety_review`).
- The three failure cases each stop with a clear message: A1 fails, A1's output is missing a column, and the A1 folder is wrong.

Not tested: a live A1 download. This workspace can't reach data.sa.gov.au, so I used a stand-in script that writes your real A1 output CSV. Do one real `--from-a1` run on your Mac.

Before it works on your machine

- **Your folder copy had gone back to the older version**, the one with `--retrain` and a fallback to the LR model. I replaced it with the always-train version plus the A1 link, which matches what you asked for. If you meant to go back to the older version, tell me.
- **`crash-risk-ingestion/` isn't in your local folder yet.** It arrives with Kieren's PR, so merge and pull first.
- **`train_models.py` still has to write `best_model.joblib`** (the snippet from earlier), or the run stops at the training check.
- **A1 and A2 need to share one environment.** That means installing both `requirements.txt` files, including geopandas and pandera for A1, plus xgboost.
- **`--a1-offline` needs an earlier online run** to fill A1's cache. As Kieren's `.gitignore` notes, a fresh clone has no cached data.

Prompt 14 (2026-10-30 05:22 UTC)

it seems as though kieren has made a pr that merges the two artefacts

Response 14

That fits the `.gitignore` you showed: `crash-risk-ingestion/` (A1) sitting next to `hybrid_ai_starter/` (A2) in one repo. The `--from-a1` option I just added expects exactly that layout, so the two should work together.

The thing to check is whether his PR also **links** the two, not just puts them in one repo. If it does, you'd have two ways of doing the same job, or a merge conflict. In the PR's "Files changed" tab, look for:

1. **Is `prolog_interface.py` changed?** If so, his version and mine will conflict, and we need to pick one or combine them.
2. **Is there a new run script?** Something like `run_all.py`, `main.py` or a `Makefile` that runs `data_ingestion.py` and then A2. That would duplicate `--from-a1`.
3. **Did `data_ingestion.py` or A1's config change?** In particular, check whether the output still goes to `crash-risk-ingestion/data/processed/intersection_sites.csv`. That's the path my interface reads.
4. **Is `train_models.py` in the list?** It's still the missing piece for `best_model.joblib`.

5. **Is there one `requirements.txt` at the top level**, or a separate one per artefact?

If none of the first three apply, his PR handles the folder layout, and your interface handles the link. That's a clean split, and it fits your role as the integrator.

The quickest way for me to check all five is for you to merge or check out his branch locally and reconnect the `HybridAI-main` folder. Then I can read the whole merged repo and run `--from-a1` against the real layout. Screenshots of the "Files changed" list would also work.

Prompt 15 (2026-10-30 05:24 UTC)

scrap kierens pr, can we just merge the directories into one so that we can then edit the prolog_interface to call on the files n stuff locally

Response 15

[Tool action: Bash — List A1 package files]

[Tool action: Bash — Compare submitted A1 with GitHub package]

[Tool action: Bash — Check A1 paths are relative]

[Tool action: Bash — Check package versions for combined requirements]

[Tool action: Bash — Prepare merged repo files and requirements]

[Tool action: Bash — Build file list for device commit]

[Tool action: device_commit_files —]

[Tool action: Bash — Run A1 tests in combined environment]

I've merged the two artefacts into your `HybridAI-main` folder. `prolog_interface.py` needed no further changes: it already looks for A1 at `../crash-risk-ingestion` by default, which is now where it sits.

```
HybridAI-main/
├── requirements.txt          (new: one install for both artefacts)
├── crash-risk-ingestion/     (A1, copied in unchanged)
│   ├── data_ingestion.py
│   ├── ingestion/  tests/  docs/
│   └── requirements.txt
└── hybrid_ai_starter/        (A2)
    ├── prolog_interface.py   (has the --from-a1 option)
    ├── train_models.py  prepare_site.py  predict_site.py
    ├── crash_risk_rules.pl
    └── processed/
```

What I put in:

- **A1:** the same code you submitted for A1. I checked it file by file against `A1_YourFAN.zip`, and it's identical. It uses relative paths, so it works in the new location. All 12 A1 tests pass in the combined environment.
- **The top-level `requirements.txt`** installs A1's packages plus scikit-learn, joblib and xgboost (3.4.1). I checked these versions work together.

Running it on your Mac:

```
cd ~/Downloads/HybridAI-main
python3 -m venv .venv && source .venv/bin/activate
```

```
pip install -r requirements.txt
brew install swi-prolog

cd hybrid_ai_starter
python3 prolog_interface.py --from-a1 --all
```

After setup, that last line is the whole pipeline from A1 to A2. The first run must be online so A1 can download the data. After that, `--from-a1 --a1-offline` reuses the cached downloads.

One blocker left: your local `train_models.py` is still the two-model version and doesn't save `best_model.joblib`. The run will fetch and validate the A1 data, then stop at the training check. Add the snippet from earlier, or ask dobi0016 to add it. If you want, I can make that change to your local copy so you can do a full test run now.

Prompt 16 (2026-10-30 05:27 UTC)

can i make a pr request for the changes in the hybridAI repo? like the other stuff

Response 16

Yes, you can. One catch first: your `HybridAI-main` folder is a ZIP download, not a Git clone. It has no `.git` folder, so you can't push from it. You'd clone the repo fresh, copy your changes in, and open the PR from there. I can't run Git or reach GitHub from here, so you'll need to run these yourself.

1. Clone and make a branch

```
cd ~/Documents
git clone <HybridAI repo URL>
cd HybridAI
git checkout -b integrate-a1-a2
```

The repo URL is the green **Code** button on GitHub. If the clone folder isn't called `HybridAI`, use its actual name.

2. Copy in your changes

```
cp -R ~/Downloads/HybridAI-main/crash-risk-ingestion .
cp ~/Downloads/HybridAI-main/requirements.txt .
cp ~/Downloads/HybridAI-main/hybrid_ai_starter/prolog_interface.py hybrid_ai_starter/
```

3. Check what changed, then commit and push

```
git status
git add crash-risk-ingestion requirements.txt hybrid_ai_starter/prolog_interface.py
git commit -m "Merge A1 ingestion into repo and link it to the A2 pipeline"
git push -u origin integrate-a1-a2
```

Before you commit, check that `git status` doesn't list any `data/`, `.venv/` or `__pycache__/` folders. A1's own `.gitignore` already excludes its `data/` folder.

4. Open the PR

GitHub will show a **"Compare & pull request"** banner for the branch you just pushed. Click it. A description along these lines would work:

- Adds Artefact 1 (`crash-risk-ingestion/`) unchanged, next to `hybrid_ai_starter/`
- Top-level `requirements.txt` for one environment (A1 + scikit-learn, joblib, xgboost)

- `prolog_interface.py`: always retrains and uses `models/best_model.joblib`; new `--from-a1` option runs A1's `data_ingestion.py`, checks its output columns and feeds it to A2
- Requires `train_models.py` to save `best_model.joblib`

Before you push, sort out these with the team:

- **Kieren's PR adds the same `crash-risk-ingestion/` folder.** If both PRs get merged, they'll conflict. Tell the team you're replacing his and ask him to close it, or the reviewer won't know which to take.
- **`train_models.py` isn't your file.** Either dobi0016 adds the `best_model.joblib` save, or you include it in this PR with their OK. Without it, the merged pipeline stops at the training step.
- **Kieren's `predict_site.py` change** (loading `best_model.joblib`) dies with his PR. Your interface sets that path at run time, so you don't need it. Running `predict_site.py` on its own would still load the LR model, though. If you want to keep it, add the one-line change to your PR.
- **A2's outputs aren't ignored yet.** There's no ignore rule for `hybrid_ai_starter/output/` or `models/*.joblib`, so don't `git add .` after a test run, or the generated files get uploaded too. Adding the files by name, as in step 3, avoids that.

Prompt 17 (2026-10-30 05:31 UTC)

https://github.com/Spyder-9999/HybridAI/tree/main is this the hybridai repo url

Response 17

That's the repo's page on GitHub, and it fits your folder: GitHub names a ZIP download `<repo>-<branch>`, so `HybridAI` on `main` gives `HybridAI-main`.

For cloning, drop the `/tree/main` part:

```
git clone https://github.com/Spyder-9999/HybridAI.git
cd HybridAI
```

This clones the `main` branch by default. Since the repo belongs to Spyder-9999, pushing a branch (`git push -u origin integrate-a1-a2`) only works if they've added you as a collaborator. If you're not one, the push will be refused. In that case, either ask them to add you, or click **Fork** on GitHub, push to your fork, and open the PR from there into `Spyder-9999/HybridAI`.

Prompt 18 (2026-10-30 05:32 UTC)

[image]

its asking for username

Response 18

GitHub is asking you to sign in, which usually means the repo is private, so you need to be signed in and have access to it.

GitHub no longer accepts your account password here. Press **Ctrl+C** to cancel, then use one of these:

Option A: GitHub CLI (easiest)

```
brew install gh
gh auth login
```

Choose **GitHub.com**, then **HTTPS**, then **Login with a web browser**. It shows a one-time code and opens a browser page where you sign in and paste the code. After that:

`gh repo clone Spyder-9999/HybridAI`

`gh auth login` also sets Git up with your sign-in, so `git push` later works without asking again.

Option B: personal access token

1. On GitHub, go to **Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token**. Tick the **repo** scope and copy the token.
2. Run the `git clone` again. For **Username**, enter `a-huss20`. For **Password**, paste the token, not your GitHub password. Nothing shows while you paste, which is normal.

Either way, don't paste the token into this chat or anywhere else.

If it still fails after signing in with an error like "Repository not found", your account doesn't have access yet. Ask Spyder-9999 to add `a-huss20` as a collaborator under the repo's **Settings → Collaborators**.

Prompt 19 (2026-10-30 05:35 UTC)

[image]

Response 19

It cloned into `~/Documents/HybridAI`.

Before copying anything in, check what's on `main` now. If Kieren's PR was already merged, `crash-risk-ingestion/` will be there, and you'd be overwriting it rather than adding it.

```
cd HybridAI
ls
git log --oneline -5
```

Then make your branch and copy your files in:

```
git checkout -b integrate-a1-a2
cp -R ~/Downloads/HybridAI-main/crash-risk-ingestion .
cp ~/Downloads/HybridAI-main/requirements.txt .
cp ~/Downloads/HybridAI-main/hybrid_ai_starter/prolog_interface.py hybrid_ai_starter/
git status
```

Paste the output of `ls`, `git log` and `git status` here before you commit, and I'll check that only the intended files are going in.

Prompt 20 (2026-10-30 05:35 UTC)

kieren has nothing to do with the merge anymore

Response 20

That makes it simpler. With his PR out of the picture, your PR is the only thing adding A1, so there's nothing to clash with. Carry on with the same steps:

```
cd ~/Documents/HybridAI
git checkout -b integrate-a1-a2
cp -R ~/Downloads/HybridAI-main/crash-risk-ingestion .
cp ~/Downloads/HybridAI-main/requirements.txt .
cp ~/Downloads/HybridAI-main/hybrid_ai_starter/prolog_interface.py hybrid_ai_starter/
git status
```

Two things that were tied to his PR now sit with yours:

- **`predict_site.py` loading `best_model.joblib`.** His one-line change goes away with his PR. Your interface doesn't need it, because it sets the model path itself. If you want `predict_site.py` to load the right model when it's run on its own, add the change to your PR: on line 29, change `logistic_regression.joblib` to `best_model.joblib`.
- **`train_models.py` saving `best_model.joblib`.** It still needs doing by someone, either dobi0016 or you in this PR with their OK.

Paste the `git status` output here and I'll check it before you commit.

Prompt 21 (2026-10-30 05:36 UTC)

so what am i doing exactly in these commands, i dont want to ruin patricks file directory

Response 21

Nothing in those commands touches Patrick's repo on GitHub. They only change your local copy in `~/Documents/HybridAI`, and even then only on a separate branch. His repo changes only if he reviews your PR and clicks Merge.

Where things live

- **Patrick's repo on GitHub** is the shared copy, and the `main` branch is what everyone uses.
- **`~/Documents/HybridAI`** is the copy you just cloned. It's a full copy on your Mac. Anything you do here stays on your Mac until you push.
- **`~/Downloads/HybridAI-main`** is the old ZIP download where I put the merged files. It isn't connected to Git at all.

What each command does

```
cd ~/Documents/HybridAI
```

Moves into your cloned copy. Nothing changes.

```
git checkout -b integrate-a1-a2
```

Creates a new **branch** and switches to it. A branch is a separate line of changes. Everything you do from here is recorded on `integrate-a1-a2`, not on `main`. You can go back to an untouched `main` at any time with `git checkout main`.

```
cp -R ~/Downloads/HybridAI-main/crash-risk-ingestion .
```

Copies the A1 folder into your clone. It's new, so it doesn't replace anything.

```
cp ~/Downloads/HybridAI-main/requirements.txt .
```

Copies the combined requirements file to the top of the repo. **This one could overwrite something:** if Patrick's repo already has a `requirements.txt` at the top level, it gets replaced. Check first with `ls` (see below).

```
cp ~/Downloads/HybridAI-main/hybrid_ai_starter/prolog_interface.py hybrid_ai_starter/
```

Replaces `prolog_interface.py` with your new version. That's your file, so it's the change you intend.

```
git status
```

Only reports. It lists which files are new or modified compared with `main`, so you can confirm nothing else changed.

Later steps (not yet)

- `git add` + `git commit` saves a snapshot on your branch, still only on your Mac.
- `git push` uploads your **branch** to GitHub as `integrate-a1-a2`. Patrick's `main` stays exactly as it is.
- **The PR** asks Patrick to review your branch. He sees every changed line and decides whether to merge it. Until he merges, his `main` is unaffected, and he can reject or ask for changes.

Before the copy commands, run this

```
ls
ls hybrid_ai_starter
```

If a `requirements.txt` already exists at the top level, or there's anything named `crash-risk-ingestion`, paste the output here first. I'll tell you whether to merge the requirements files instead of overwriting.

If anything goes wrong locally, you can undo it all with `git checkout main` followed by `git branch -D integrate-a1-a2`, or just delete the `HybridAI` folder and clone again. None of this affects GitHub.

Prompt 22 (2026-10-30 05:38 UTC)

[image]

Response 22

That's what we wanted, and nothing of Patrick's was overwritten:

- **`modified: hybrid_ai_starter/prolog_interface.py`**: your file, the intended change.
- **Untracked `crash-risk-ingestion/` and `requirements.txt`**: "untracked" means Git has never seen them, so they didn't exist in the repo before. Adding them doesn't replace anything.

No other files are listed, so nothing else in the repo changed.

Stage the files and check the list

```
git add crash-risk-ingestion requirements.txt hybrid_ai_starter/prolog_interface.py
git status
```

`git status` should now show about 21 files in green under "Changes to be committed": 19 inside `crash-risk-ingestion/`, plus `requirements.txt` and `prolog_interface.py`. Make sure nothing in that list is under `data/`, `__pycache__/` or `.venv/`. If anything looks off, `git restore --staged <file>` unstages it without deleting it.

Commit and push

```
git commit -m "Add Artefact 1 ingestion and link it to the A2 pipeline via prolog_interfa
ce --from-a1"
git push -u origin integrate-a1-a2
```

The push uploads your branch only; `main` stays as it is. Then open the PR from the "Compare & pull request" banner on the repo page. If the push is refused with a permission error, Patrick needs to add `a-huss20` as a collaborator, or you can push to a fork instead.

Prompt 23 (2026-10-30 05:39 UTC)

[image]

what do i do now

Response 23

The push worked. Your branch is now on GitHub as `integrate-a1-a2`, and `main` is unchanged. The 21 files match what we expected. The 347 deletions are all in `prolog_interface.py`, where your new version replaces the old one.

Open the PR

1. Open the link from your terminal: **https://github.com/Spyder-9999/HybridAI/pull/new/integrate-a1-a2**
2. Check the top of the page reads **base: `main` ← compare: `integrate-a1-a2`**.
3. Add a title and description, for example:

Title: Integrate Artefact 1 ingestion with the A2 hybrid pipeline

Description:

```
What this adds
- crash-risk-ingestion/: Artefact 1 data pipeline, unchanged from the A1 submission
- requirements.txt (repo root): one environment for both artefacts
  (A1 packages + scikit-learn, joblib, xgboost)
- hybrid_ai_starter/prolog_interface.py:
  - runs train_models.py on every run and uses models/best_model.joblib
    (any old best_model.joblib is deleted first, so a stale model is never used)
  - new --from-a1 option: runs A1's data_ingestion.py, checks its output has
    every column A2 needs, then copies intersection_sites.csv into processed/
  - --a1-offline reuses A1's cached downloads; --a1-dir points at another A1 folder
  - fact predicates declared dynamic, so sites with no traffic volume no longer
    crash Prolog

How to run
  pip install -r requirements.txt   (plus SWI-Prolog: brew install swi-prolog)
  cd hybrid_ai_starter
  python3 prolog_interface.py --from-a1 --all

Needed before this works end to end
- train_models.py must save the selected model to models/best_model.joblib
```

4. Under **Reviewers** on the right, add Patrick (Spyder-9999) and any other teammates.
5. Click **Create pull request**.

After that

- Tell the team in your chat that the PR is up, and that `train_models.py` still needs the `best_model.joblib` save before the full pipeline runs.

- If someone asks for changes, edit the files in `~/Documents/HybridAI` (still on the `integrate-a1-a2` branch), then run `git add`, `git commit` and `git push` again. The PR updates automatically, so you don't open a new one.
- Patrick merges it when he's happy. Until then, `main` stays as it is.

Prompt 24 (2026-10-30 05:41 UTC)

so how can i get patrick to accept my pr request

Response 24

You can't approve it yourself, so the job is to make it quick and safe for him to say yes. The main thing that could hold it up is the `train_models.py` gap.

1. Fix the `train_models.py` gap first

As things stand, if Patrick merges your PR, `prolog_interface.py` stops at the training step, because `train_models.py` doesn't save `best_model.joblib`. A reviewer who runs it and sees an error will hesitate. Do one of these:

- Ask dobi0016 to add the `best_model.joblib` save (the snippet from earlier) and merge that first, or
- Get their OK and add it to your branch yourself, so your PR works on its own when merged.

The second option is quicker with the deadline on 5 Nov.

2. Show that it works

Once `train_models.py` is sorted, run it from your clone:

```
cd ~/Documents/HybridAI/hybrid_ai_starter
python3 prolog_interface.py --from-a1 --all
```

Paste the end of the output (the action counts) as a comment on the PR. A reviewer is far more likely to merge something they can see working.

3. Message him directly

Don't rely on the GitHub notification alone. Something like:

> Hey Patrick, PR is up: integrate-a1-a2. It adds A1 (crash-risk-ingestion) unchanged, one requirements.txt for both, and updates my prolog_interface.py so the whole thing runs from A1 to A2 with one command: `python3 prolog_interface.py --from-a1 --all`. It only adds new files plus my own prolog_interface.py, so nothing else in the repo changes. Tested end to end, output's in the PR comments. Could you review and merge when you get a chance? Due Wednesday.

That message answers his likely questions up front: what changed, whether it touches anyone else's files, whether it works, and why now.

4. Respond quickly to feedback

If he asks for changes, make them on the same branch and push. The PR updates itself. Reply to each comment so he can see it's been dealt with.

If anything in his review comments isn't clear, paste them here and I'll help you work through them.

Prompt 25 (2026-10-30 05:46 UTC)

so as it stands, does my prolog_interface currently call on the data_ingestion stuff? can it run on 1 command

Response 25

Yes, but only when you add `--from-a1`. Without it, the interface skips A1 and uses whatever `intersection_sites.csv` is already in `processed/`.

The one command, after the one-time setup (`pip install -r requirements.txt` and `brew install swi-prolog`):

```
cd ~/Documents/HybridAI/hybrid_ai_starter
python3 prolog_interface.py --from-a1 --all
```

That runs, in order:

1. **A1:** `crash-risk-ingestion/data_ingestion.py` fetches, validates and transforms the data, and writes `intersection_sites.csv`.
2. **Handoff:** the interface checks that the CSV has every column A2 needs and copies it into `hybrid_ai_starter/processed/`.
3. **Training:** `train_models.py` trains and compares the models.
4. **Prepare and predict:** the best model scores each site.
5. **Prolog:** the rules decide each site's action and reason.
6. **Results:** they print and are saved to `output/`.

As it stands, it stops at step 3. The `train_models.py` in the repo doesn't save `best_model.joblib`, so the run gets through A1 and the handoff, then fails with "did not save best_model.joblib". Once that file saves the best model, the one command runs from start to finish.

Swap `--all` for `--site-key signal:TS:025` to do one site, or drop it to get the search menu. Add `--a1-offline` to reuse A1's cached downloads after the first online run.

Prompt 26 (2026-10-31 05:48 UTC)

so how does patrick accept my pull / pr?

Response 26

Patrick does it on GitHub in a few clicks. Nothing for him to install or run.

What Patrick does

1. Opens **github.com/Spyder-9999/HybridAI** and clicks the **Pull requests** tab.
2. Clicks your PR ("Integrate Artefact 1 ingestion…").
3. Optionally reviews it:
   - The **Files changed** tab shows every added or changed line.
   - **Review changes** (top right) lets him pick **Approve**, **Comment** or **Request changes**.
4. Goes back to the **Conversation** tab, scrolls to the bottom, and clicks the green **Merge pull request** button, then **Confirm merge**.
5. Optionally clicks **Delete branch** to tidy up `integrate-a1-a2`. Your changes stay in `main`.

Your changes are now part of `main`.

If the button isn't green

- **"This branch has conflicts"**: someone changed the same files on `main` since you branched. Tell me what it lists and I'll help you fix it on your branch.
- **"Review required"**: the repo is set up to need an approval first. Patrick (or whoever's allowed) clicks **Approve**, then the merge button unlocks.

Since your PR only adds new files and changes your own `prolog_interface.py`, a conflict is unlikely unless someone else edited that file.

After it's merged

Everyone, including you, updates their local copy:

```
cd ~/Documents/HybridAI
git checkout main
git pull
```

Prompt 27 (2026-10-31 05:51 UTC)

so does kieren need to edit the train_model now?

Response 27

Yes, someone needs to, and if Kieren is the team's ML person, it's him. The file's header names the author as dobi0016. If that's Kieren's FAN, it's his file; if not, check with whoever dobi0016 is.

Why it's needed: your PR depends on `train_models.py` saving `models/best_model.joblib`. The version on `main` only saves `logistic_regression.joblib` and `decision_tree.joblib`, so `prolog_interface.py` stops at the training step. When Kieren said "point it at best_model.joblib", he was likely counting on his own PR to create that file. With his PR scrapped, nothing on `main` creates it.

What he needs to add

1. **Required:** at the end of `main()`, pick the best model and save it as `models/best_model.joblib`. That's the snippet I gave earlier, which picks the winner by CV F1.
2. **For the four-model comparison:** add Random Forest and XGBoost to the `models` dictionary, and `xgboost` to `requirements.txt`. Your `requirements.txt` already includes it.
3. **Don't change the feature lists**, unless `predict_site.py` gets the same change.

Two ways to get it done

- **Kieren makes a small PR** with just `train_models.py`. It should be merged before or together with yours.
- **You add it to your branch** with his OK, so your PR works on its own. This is faster with Wednesday's deadline.

Something like this to send him:

> Hey, since your PR's scrapped, nothing on main writes models/best_model.joblib anymore. My PR needs train_models.py to save the best model there. Can you add that (plus RF/XGBoost if that's the 4-model version), or are you OK with me adding it to my PR?

Prompt 28 (2026-10-31 05:51 UTC)

how can i show you the current one

Response 28

[Tool action: device_request_folder_access — [‘~/Documents/HybridAI’]]

[Tool action: device_list_dir — /Users/aftabhussaini/Documents/HybridAI/hybrid_ai_starter]

[Tool action: device_stage_files — [‘/Users/aftabhussaini/Documents/HybridAI/hybrid_ai_starter/train_models.py’, ‘/Users/aftabhussaini/Documents/HybridAI/hybrid_ai_starter/predict_site.py’]]

[Tool action: Bash — Check current train_models and predict_site]

[Tool action: device_list_dir — /Users/aftabhussaini/Documents/HybridAI/.git/refs/remotes/origin]

[Tool action: device_stage_files — [‘/Users/aftabhussaini/Documents/HybridAI/.git/packed-refs’]]

[Tool action: Bash — List remote branches in clone]

I’ve connected your `~/Documents/HybridAI` folder and checked it directly.

The `train_models.py` in your clone is byte-for-byte the old two-model version. It’s the same file as `main`, since your branch started from `main`.

- It trains **Logistic Regression and Decision Tree only**. There’s no Random Forest or XGBoost.
- It saves `logistic_regression.joblib` and `decision_tree.joblib`, and **nothing writes `best_model.joblib`**.
- `predict_site.py` on `main` still loads `logistic_regression.joblib` and labels the model “Logistic Regression”.

So yes, `train_models.py` still needs editing before your one command runs end to end.

The repo also has a branch called **`merge-part1-part2`** besides `main` and yours. That’s very likely Kieren’s scrapped PR. If his four-model `train_models.py` exists anywhere, it’s probably there. I can’t run Git on your Mac, but you can check from your `HybridAI` folder:

```
git diff --stat main origin/merge-part1-part2
git show origin/merge-part1-part2:hybrid_ai_starter/train_models.py | grep -n "best_model
\|Forest\|XGB"
```

The first command lists which files that branch changed. The second shows whether its `train_models.py` saves `best_model.joblib` and includes RF/XGBoost. If those lines show up, the work is already done and only needs pulling into a PR. If nothing shows, nobody has written it yet. Paste the output here either way.

Prompt 29 (2026-10-31 05:56 UTC)

[image]

did it merge into the repo?

Response 29

No, it hasn’t merged.

- **The page is showing your branch, not `main`.** The branch dropdown on the left says `integrate-a1-a2`, so the files listed (`crash-risk-ingestion`, `requirements.txt`) are your branch’s files.
- **“This branch is 1 commit ahead of `main`”** means your commit isn’t in `main` yet. After a merge, this would say the branch is up to date with `main`.

- **The yellow "Compare & pull request" banner** usually means there's no open PR for this branch yet. The "Pull requests 1" at the top is likely a different PR, probably Kieren's `merge-part1-part2`.

What to do

1. Click **Compare & pull request** in the yellow banner.
2. Check it reads **base: `main` ← compare: `integrate-a1-a2`**.
3. Paste in the title and description from earlier, add Spyder-9999 as a reviewer, and click **Create pull request**.

The **Pull requests** count should then go to 2. If Kieren's PR is the other one, it's worth asking him to close it so Patrick doesn't merge both.

You can check whether `main` has your changes at any time by switching the dropdown to `main`. If `crash-risk-ingestion` is missing there, it hasn't been merged.

Prompt 30 (2026-10-31 05:57 UTC)

[image]

Response 30

This is the right screen. **"Able to merge"** means there are no conflicts with `main`, so Patrick will get the green merge button.

GitHub split your commit message across the title and description boxes. Clear both and paste these in.

Title:

```
Integrate Artefact 1 ingestion with the A2 hybrid pipeline
```

Description:

```
## What this adds
- `crash-risk-ingestion/`: Artefact 1 data pipeline, unchanged from the A1 submission
- `requirements.txt` (repo root): one environment for both artefacts (A1 packages + sciki
t-learn, joblib, xgboost)
- `hybrid_ai_starter/prolog_interface.py`:
  - runs `train_models.py` on every run and uses `models/best_model.joblib` (any old copy
is deleted first, so a stale model is never used)
  - new `--from-a1` option: runs A1's `data_ingestion.py`, checks its output has every co
lumn A2 needs, then copies `intersection_sites.csv` into `processed/`
  - `--a1-offline` reuses A1's cached downloads; `--a1-dir` points at another A1 folder
  - Prolog fact predicates declared dynamic, so sites with no traffic volume no longer cr
ash the run

Only new files are added, plus my own `prolog_interface.py`. No one else's files are chan
ged.

## How to run
pip install -r requirements.txt
brew install swi-prolog
cd hybrid_ai_starter
python3 prolog_interface.py --from-a1 --all
```

```
## Needed before this runs end to end
`train_models.py` must save the selected model to `models/best_model.joblib`. The current version only saves logistic_regression.joblib and decision_tree.joblib.
```

Click **Preview** to check the formatting. Then, in the right-hand sidebar, set **Reviewers** to Spyder-9999 and click **Create pull request**.

Prompt 31 (2026-10-31 06:00 UTC)

so after kieren edits the train_models file we should be all good right? we can run the entire program with prolog_interface?

Response 31

Yes. Once Kieren's `train_models.py` change and your PR are both merged, one command runs the whole thing:

```
python3 prolog_interface.py --from-a1 --all
```

Before you rely on it, check these:

1. **What Kieren's edit needs to do.** It must:
   - save the winner to exactly `models/best_model.joblib`
   - save it as the full pipeline (preprocessing plus model)
   - leave the feature lists unchanged, or update `predict_site.py` to match
   - run without asking for input

   Send me his file when it's done and I'll check it.
2. **Both PRs are merged, and you've pulled `main`** (`git checkout main && git pull`).
3. **One-time setup on each machine that runs it:** `pip install -r requirements.txt` from the repo root, and `brew install swi-prolog`.
4. **The first run is online,** so A1 can download the source data. After that, `--a1-offline` works from the cache.

Test it yourself once before Wednesday. I've tested every stage here except one: a real A1 download from data.sa.gov.au, which this workspace can't reach. For the A1 step, I used a stand-in script that writes your real A1 output, and the handoff, training, prediction and Prolog all worked. So do one full live run on your Mac after both merges. If it passes, you're done. If anything errors, paste the output here.

Prompt 32 (2026-10-31 06:03 UTC)

kieren updated it, this is the new train_models

document: “”” Student Name: Kieren Roberts Student FAN: robe0980 File: train_models.py Date: 3/11/2026 Description: Reads the data contract, selects leakage-free features, then trains, evaluates and compares any_fsi classifiers. License: MIT Usage: python3 train_models.py “””

import json import warnings from pathlib import Path

import joblib import numpy as np import pandas as pd

from sklearn.compose import ColumnTransformer from sklearn.ensemble import RandomForestClassifier from sklearn.impute import SimpleImputer from sklearn.linear_model import LogisticRegression from sklearn.metrics import (accuracy_score, average_precision_score, classification_report, confusion_matrix, f1_score, precision_score, recall_score, roc_auc_score,) from sklearn.model_selection import (StratifiedKFold, cross_validate, train_test_split,) from

sklearn.pipeline import Pipeline from sklearn.preprocessing import OneHotEncoder, StandardScaler from sklearn.tree import DecisionTreeClassifier from xgboost import XGBClassifier

---

1. File locations and settings

BASE_DIR = Path(**file**).resolve().parent PROJECT_DIR = BASE_DIR.parent

CONTRACT_FILE = PROJECT_DIR / “crash-risk-ingestion” / “docs” / “data_contract.json” DATA_FILE = BASE_DIR / “processed” / “intersection_sites.csv” MODEL_DIR = BASE_DIR / “models” METRICS_DIR = BASE_DIR / “output”

Table in the contract that DATA_FILE corresponds to.

TABLE = “intersection_sites” TARGET = “any_fsi”

RANDOM_STATE = 42 TEST_SIZE = 0.25 CV_FOLDS = 5

Cross-validated metric used to rank the models and choose the best one.

ROC-AUC does not depend on a decision threshold, which suits a small,

imbalanced dataset.

SELECTION_METRIC = “roc_auc”

---

2. Excluded features

---

any_fsi is 1 when the site had at least one fatal or serious injury

crash. Every column built from the crash records (counts, casualties,

injuries, crash types, rates) either encodes the target directly or is

only known after the crashes happened, so all of them are excluded.

Prefix rules catch these columns even if the contract adds more later.

LEAKAGE_PREFIXES = (“cse_”, # crash severity counts (pdo, inj, si, fat, minor_inj) “cty_”, # crash type counts “uty_”, # road-user type counts (bicycle, pedestrian) “lco_”, # lighting condition counts “total_”, # total crashes / casualties / fatalities / serious injuries “fsi_”, # fsi_crashes, fsi_rate_per_mev “crash_”, # crash_rate_per_mev)

LEAKAGE_COLUMNS = { “any_fsi”, “max_severity”, “max_severity_label”, “n_crash_locations”, # number of crash points matched to the site “night_share”, # derived from lco_night / total_crashes “vru_involved”, # derived from uty_* counts }

Identifiers and free text: unique per site, so they cannot generalise.

IDENTIFIER_COLUMNS = { “site_key”, “site_id”, “site_desc”, “unique_loc”, “segment_id”, “intersection_site_key”, “geometry”, }

Exposure columns that are an exact rescaling of another feature

(million_entering_vehicles = entering_aadt x 365 x 5 / 1e6).

REDUNDANT_COLUMNS = { “million_entering_vehicles”, }

Contract dtypes that are treated as categorical features.

CATEGORICAL_DTYPES = {“str”, “string”, “object”, “category”}

def is_excluded(column): “““Return the reason a column is excluded, or None if it is usable.”“”

```
if column in LEAKAGE_COLUMNS or column.startswith(LEAKAGE_PREFIXES):
    return "outcome / leakage"
if column in IDENTIFIER_COLUMNS:
    return "identifier"
if column in REDUNDANT_COLUMNS:
    return "redundant"
return None
```

---

3. Load and validate data

---

def load_contract(): """Load the table definition for TABLE from the data contract."""

```
if not CONTRACT_FILE.is_file():
    raise FileNotFoundError(f"Data contract not found: {CONTRACT_FILE}")

contract = json.loads(CONTRACT_FILE.read_text(encoding="utf-8"))
tables = contract.get("tables", {})

if TABLE not in tables:
    raise ValueError(
        f"Table {TABLE!r} is not in the data contract. "
        f"Available tables: {', '.join(tables)}"
    )

table = tables[TABLE]
print(f"Data contract v{contract.get('contract_version')} "
      f"({contract.get('crash_period')}), table: {TABLE}")
print(f"Grain: {table.get('grain')}")
return table
```

def select_features(table): """Pick numeric and categorical features from the contract columns."""

```
expected_rows = table.get("rows")
numeric, categorical, excluded = [], [], {}

for column in table["columns"]:
    name = column["name"]
    reason = is_excluded(name)

    # A column that is entirely null in the contract carries no signal.
    if reason is None and expected_rows and column.get("nulls") == expected_rows:
        reason = "all null"

    if reason:
        excluded[name] = reason
    elif column["dtype"] in CATEGORICAL_DTYPES:
        categorical.append(name)
```

```
    else:
        numeric.append(name)

return numeric, categorical, excluded
```

def load_dataset(table, numeric, categorical): “””Load DATA_FILE and check it against the contract.”””

```
if not DATA_FILE.is_file():
    raise FileNotFoundError(
        f"Dataset not found: {DATA_FILE}\n"
        f"Check that {DATA_FILE.name} is in the processed folder."
    )

data = pd.read_csv(DATA_FILE, encoding="utf-8-sig")

# Every contract column must be present.
contract_columns = [column["name"] for column in table["columns"]]
missing = [column for column in contract_columns if column not in data.columns]
if missing:
    raise ValueError("Dataset is missing contract columns: " + ", ".join(missing))

extra = [column for column in data.columns if column not in contract_columns]
if extra:
    print("Warning: columns not in the contract are ignored:", ", ".join(extra))

# Row count and primary key.
expected_rows = table.get("rows")
if expected_rows and len(data) != expected_rows:
    print(f"Warning: contract expects {expected_rows} rows, found {len(data)}.")

primary_key = table.get("primary_key")
if primary_key and data[primary_key].duplicated().any():
    raise ValueError(f"Primary key {primary_key!r} contains duplicates.")

# Numeric features must be numeric; anything unparsable becomes NaN.
for column in numeric:
    before = data[column].isna().sum()
    data[column] = pd.to_numeric(data[column], errors="coerce")
    coerced = data[column].isna().sum() - before
    if coerced:
        print(f"Warning: {coerced} non-numeric value(s) in {column} set to missing.")

for column in categorical:
    data[column] = data[column].astype("object").where(data[column].notna(), np.nan)

# Nulls above the contract's documented count suggest an upstream change.
null_limits = {column["name"]: column.get("nulls", 0) for column in table["columns"]}
for column in numeric + categorical:
    found = int(data[column].isna().sum())
    if found > null_limits.get(column, 0):
        print(f"Warning: {column} has {found} nulls (contract: {null_limits[column]}).")

# Target: binary 0/1 with enough examples of each class for CV.
data[TARGET] = pd.to_numeric(data[TARGET], errors="coerce")
data = data.dropna(subset=[TARGET]).copy()
```

```
if not data[TARGET].isin([0, 1]).all():
    raise ValueError(f"{TARGET} must contain only 0 or 1.")

data[TARGET] = data[TARGET].astype(int)
counts = data[TARGET].value_counts()

if len(counts) != 2:
    raise ValueError("Training requires examples of both target classes.")
if counts.min() < CV_FOLDS:
    raise ValueError(
        f"Each target class needs at least {CV_FOLDS} examples for "
        f"{CV_FOLDS}-fold cross-validation."
    )

return data
```

def drop_constant_features(data, numeric, categorical, excluded): “””Remove features with a single value in the data (e.g. site_type).”””

```
for features in (numeric, categorical):
    for column in list(features):
        if data[column].nunique(dropna=True) <= 1:
            features.remove(column)
            excluded[column] = "constant"
```

---

4. Preprocessing and pipeline

---

def build_preprocessor(numeric, categorical): “””Impute and scale numeric features; impute and one-hot categoricals.”””

```
numeric_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median", add_indicator=True)),
    ("scaler", StandardScaler()),
])

categorical_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="constant", fill_value="missing")),
    ("encoder", OneHotEncoder(handle_unknown="ignore")),
])

return ColumnTransformer(transformers=[
    ("numeric", numeric_pipeline, numeric),
    ("categorical", categorical_pipeline, categorical),
])
```

def build_models(y_train): “””Candidate classifiers. Each handles the class imbalance.”””

```
negatives, positives = np.bincount(y_train)

return {
    "logistic_regression": LogisticRegression(
        max_iter=2000,
        class_weight="balanced",
        random_state=RANDOM_STATE,
    ),
    "decision_tree": DecisionTreeClassifier(
        max_depth=5,
        min_samples_leaf=5,
        class_weight="balanced",
        random_state=RANDOM_STATE,
    ),
    "random_forest": RandomForestClassifier(
        n_estimators=300,
        max_depth=8,
        min_samples_leaf=3,
        class_weight="balanced",
        n_jobs=-1,
        random_state=RANDOM_STATE,
    ),
    "xgboost": XGBClassifier(
        n_estimators=200,
        max_depth=3,
        learning_rate=0.1,
        subsample=0.8,
        colsample_bytree=0.8,
        scale_pos_weight=negatives / positives,
        eval_metric="logloss",
        n_jobs=1,
        random_state=RANDOM_STATE,
    ),
}
```

def build_pipeline(model, numeric, categorical): “““Chain the preprocessor and a classifier into one saved object.”””

```
return Pipeline(steps=[
    ("preprocessor", build_preprocessor(numeric, categorical)),
    ("classifier", model),
])
```

---

5. Iterate over and evaluate models

CV_SCORING = [“accuracy”, “precision”, “recall”, “f1”, “roc_auc”, “average_precision”]

def cross_validate_model(pipeline, X_train, y_train): """Stratified k-fold CV on the training split only."""

```
folds = StratifiedKFold(n_splits=CV_FOLDS, shuffle=True, random_state=RANDOM_STATE)

with warnings.catch_warnings():
    warnings.simplefilter("ignore", category=UserWarning)
    scores = cross_validate(
        pipeline, X_train, y_train,
        cv=folds, scoring=CV_SCORING, n_jobs=-1,
    )

return {
    f"cv_{metric}_{stat}": float(getattr(scores[f"test_{metric}"], stat)())
    for metric in CV_SCORING
    for stat in ("mean", "std")
}
```

def test_metrics(pipeline, X_test, y_test): """Score a fitted pipeline on the held-out test split."""

```
predictions = pipeline.predict(X_test)
probabilities = pipeline.predict_proba(X_test)[:, 1]

return {
    "test_accuracy": float(accuracy_score(y_test, predictions)),
    "test_precision": float(precision_score(y_test, predictions, zero_division=0)),
    "test_recall": float(recall_score(y_test, predictions, zero_division=0)),
    "test_f1": float(f1_score(y_test, predictions, zero_division=0)),
    "test_roc_auc": float(roc_auc_score(y_test, probabilities)),
    "test_average_precision": float(average_precision_score(y_test, probabilities)),
    "confusion_matrix": confusion_matrix(y_test, predictions, labels=[0, 1]).tolist(),
    "classification_report": classification_report(
        y_test, predictions, labels=[0, 1],
        target_names=["no_fsi", "any_fsi"],
        output_dict=True, zero_division=0,
    ),
}
```

def evaluate_model(name, pipeline, split, features): """Cross-validate, fit, test and save one model and its metrics."""

```
X_train, X_test, y_train, y_test = split
print(f"\nTraining {name}...")

cv_scores = cross_validate_model(pipeline, X_train, y_train)
pipeline.fit(X_train, y_train)
scores = test_metrics(pipeline, X_test, y_test)

metrics = {
    "model": name,
    "target": TARGET,
    "table": TABLE,
    "features": features,
    "training_rows": int(len(X_train)),
    "test_rows": int(len(X_test)),
    **cv_scores,
    **scores,
```

```
}

model_path = MODEL_DIR / f"{name}.joblib"
joblib.dump(pipeline, model_path)

metrics_path = METRICS_DIR / f"{name}_metrics.json"
metrics_path.write_text(json.dumps(metrics, indent=2) + "\n", encoding="utf-8")

print(f"  {CV_FOLDS}-fold CV ROC-AUC: {cv_scores['cv_roc_auc_mean']:.3f} "
      f"+/- {cv_scores['cv_roc_auc_std']:.3f}")
print(f"  {CV_FOLDS}-fold CV F1:      {cv_scores['cv_f1_mean']:.3f} "
      f"+/- {cv_scores['cv_f1_std']:.3f}")
print(f"  Test ROC-AUC: {scores['test_roc_auc']:.3f}   "
      f"Test F1: {scores['test_f1']:.3f}   "
      f"Test recall: {scores['test_recall']:.3f}")
print(f"  Confusion matrix: {scores['confusion_matrix']}")
print(f"  Saved model:   {model_path}")
print(f"  Saved metrics: {metrics_path}")

return metrics
```

---

6. Train and compare models

COMPARISON_COLUMNS = [“model”, “cv_roc_auc_mean”, “cv_roc_auc_std”, “cv_average_precision_mean”, “cv_f1_mean”, “cv_f1_std”, “test_accuracy”, “test_precision”, “test_recall”, “test_f1”, “test_roc_auc”, “test_average_precision”,]

def main(): “““Run the complete training and comparison process.”””

```
MODEL_DIR.mkdir(parents=True, exist_ok=True)
METRICS_DIR.mkdir(parents=True, exist_ok=True)

table = load_contract()
numeric, categorical, excluded = select_features(table)
data = load_dataset(table, numeric, categorical)
drop_constant_features(data, numeric, categorical, excluded)
features = numeric + categorical

if not features:
    raise ValueError("No usable features remain after exclusions.")

print(f"\nRows with known target: {len(data)}")
print("Target counts:")
print(data[TARGET].value_counts().sort_index().to_string())

print(f"\nPredictor features ({len(features)}):")
for feature in features:
```

```
    kind = "categorical" if feature in categorical else "numeric"
    print(f"  - {feature} ({kind})")

print(f"\nExcluded columns ({len(excluded)}):")
for reason in sorted(set(excluded.values())):
    names = [name for name, why in excluded.items() if why == reason]
    print(f"  {reason}: {', '.join(names)}")

X = data[features].copy()
y = data[TARGET].copy()

split = train_test_split(
    X, y,
    test_size=TEST_SIZE,
    random_state=RANDOM_STATE,
    stratify=y,
)

all_metrics = []
for name, model in build_models(split[2]).items():
    pipeline = build_pipeline(model, numeric, categorical)
    all_metrics.append(evaluate_model(name, pipeline, split, features))

# Rank on cross-validation (training data) rather than the single test
# split, so the choice is not tuned to the test set.
comparison = (
    pd.DataFrame(all_metrics)[COMPARISON_COLUMNS]
    .sort_values(f"cv_{SELECTION_METRIC}_mean", ascending=False)
    .reset_index(drop=True)
)
best_name = comparison.loc[0, "model"]

joblib.dump(joblib.load(MODEL_DIR / f"{best_name}.joblib"),
            MODEL_DIR / "best_model.joblib")

summary = {
    "table": TABLE,
    "target": TARGET,
    "selection_metric": f"cv_{SELECTION_METRIC}_mean",
    "best_model": best_name,
    "features": features,
    "excluded_features": excluded,
    "models": comparison.to_dict(orient="records"),
}
comparison_path = METRICS_DIR / "model_comparison.json"
comparison_path.write_text(json.dumps(summary, indent=2) + "\n", encoding="utf-8")

print("\nModel comparison (ranked by "
      f"{CV_FOLDS}-fold CV {SELECTION_METRIC})")
print("-" * 60)
print(comparison.round(3).to_string(index=False))
print(f"\nBest model: {best_name} -> {MODEL_DIR / 'best_model.joblib'}")
print(f"Comparison saved to: {comparison_path}")
print("\nTraining and evaluation complete.")
```

if **name** == "**main**": main()

Response 32

Kieren's version does save `best_model.joblib`. But it now picks its features from A1's data contract instead of a fixed list, which could break how prediction reads them. Let me run it against the real files.

[Tool action: Bash — Set up test copy and find feature usage]

Now save Kieren's file into the test copy and run it:

[Tool action: Write — /tmp/claude-0/-home-claude/5e1f276b-c718-5557-a729-bdbd411a9f4e/scratchpad/kieren_train_models.py]

[Tool action: Bash — Run Kieren's train_models on real data]

[Tool action: Bash — Check feature match and run full pipeline]

Two incompatibilities found. Let me look at `resp` and the code paths that need fixing.

[Tool action: Bash — Inspect resp column and prediction code]

[Tool action: Bash — Make prediction follow the model's trained features]

[Tool action: Bash — Rerun full pipeline with Kieren's trainer]

Everything passes now. Updating the file in your clone:

[Tool action: Bash — Stage fixed interface for delivery]

[Tool action: device_commit_files —]

Almost. Kieren's file is good, but it broke two things in my `prolog_interface.py`. I've fixed both in your `~/Documents/HybridAI` clone and run everything again with his file.

Kieren's `train_models.py` works

- It saves `models/best_model.joblib` as the full pipeline (preprocessing plus model), which is what's needed.
- It compares LR, DT, Random Forest and XGBoost, and picks the winner by cross-validated ROC-AUC on the training data, not the test set.
- On the current data, **Random Forest wins** (CV ROC-AUC 0.679, ahead of LR at 0.673).

What broke, and why

1. **Prediction failed with "columns are missing: {'resp'}".** Kieren now picks features from A1's data contract instead of a fixed list. That adds `resp` (whether SA Government or the local council is responsible for the road) and drops `road_no` (always empty) and `site_type` (the same value at every site). `predict_site.py` still builds its input from the old fixed list, so the new model got data without `resp`.
   - **Fix:** the interface now builds the model's input from the saved model's own record of the columns it was trained on. Prediction always matches whatever features `train_models.py` picks, even if Kieren changes them again.
2. **`--from-a1` crashed.** My A1 output check read a `FEATURES` list from `train_models.py`, and his version no longer has one.
   - **Fix:** it now checks A1's output against the data contract, the same file Kieren's script reads, plus the fields `prepare_site.py` needs.

Retested with his file:

- `--from-a1 --site-key signal:TS:025`: Random Forest gives P = 77.0%, and Prolog returns `priority_safety_review`.
- `--all`: all 677 sites are assessed.
- A wrong A1 folder still fails with a clear message.

Push the fix to your PR

```
cd ~/Documents/HybridAI
git add hybrid_ai_starter/prolog_interface.py
git commit -m "Predict with the model's trained feature columns; check A1 output against
the data contract"
git push
```

The PR updates itself. No new PR needed.

Things to tell the team

- **Merge order:** Kieren's script reads `../crash-risk-ingestion/docs/data_contract.json`, which only exists once your PR is merged. Merge yours first, or both together.
- **Running `predict_site.py` on its own** will fail with the new model, the same `resp` error, and it still loads `logistic_regression.joblib`. That doesn't affect the one-command pipeline, which now does prediction itself. If anyone runs it alone in the demo, its feature list needs `resp` added.
- **The live A1 download** is still the one stage I couldn't test from here. Do one full `--from-a1 --all` run on your Mac after the merges.

Prompt 33 (2026-11-03 06:11 UTC)

[image]

Response 33

> [Tool action: device_list_dir — /Users/aftabhussaini/Documents/HybridAI/hybrid_ai_starter]
>
> [Tool action: device_commit_files —]
>
> [Tool action: device_list_dir — /Users/aftabhussaini/Documents/HybridAI/hybrid_ai_starter]

Git found nothing to commit because the fixed file wasn't in your folder. When I checked, `prolog_interface.py` was back to the previous version (31,192 bytes), even though I'd written the new one (33,238 bytes). Something replaced it after my write, maybe an editor that had the old file open and saved it. If you have `prolog_interface.py` open in VS Code or similar, close it without saving.

I've written the fixed version again. It's 33,238 bytes in your folder now. Run:

```
cd ~/Documents/HybridAI
git diff --stat
```

This time it should list `hybrid_ai_starter/prolog_interface.py` as changed. Then:

```
git add hybrid_ai_starter/prolog_interface.py
git commit -m "Predict with the model's trained feature columns; check A1 output against
the data contract"
git push
```

If `git diff --stat` shows nothing again, tell me before you do anything else. That means something on your Mac is still restoring the old file.

Prompt 34 (2026-11-03 06:16 UTC)

[image]

Response 34

That worked. The push went through (`dbb60c7..d766f1c`), so your PR now has the fix, and Patrick will see it on the same PR.

The identity warning is harmless but worth fixing. Git didn't know your name and email, so it used your Mac's hostname (`aftabhussaini@Aftabs-MacBook-Air.local`). GitHub can't match that address to your account, so this commit won't show your profile on the PR. Set it once for future commits:

```
git config --global user.name "Aftab Hussaini"
git config --global user.email "<the email on your GitHub account>"
```

Leave the commit you already pushed as it is. Changing it would mean rewriting history and force-pushing, which isn't worth it.

Where things stand:

- **Your PR:** A1 folder, combined `requirements.txt`, and the updated `prolog_interface.py`. It's tested with Kieren's four-model `train_models.py`.
- **Kieren's `train_models.py`:** needs to be on `main`, or in your PR, before the pipeline runs end to end. It reads A1's data contract from `crash-risk-ingestion/`, so merge your PR first or both together.
- **After both are merged:** pull `main` and do one live run on your Mac:

```
cd ~/Documents/HybridAI
git checkout main && git pull
pip install -r requirements.txt
cd hybrid_ai_starter
python3 prolog_interface.py --from-a1 --all
```

Prompt 35 (2026-11-03 06:17 UTC)

i dont see it in the pull requests

Response 35

Most likely the PR was never actually created. The form you had open only becomes a PR once you click the green **Create pull request** button at the bottom. Pushing commits to the branch doesn't create one.

Check from your terminal (you're already signed in to `gh`):

```
gh pr list --state all
```

- **If `integrate-a1-a2` isn't listed**, create the PR now:

```
gh pr create --base main --head integrate-a1-a2 \
  --title "Integrate Artefact 1 ingestion with the A2 hybrid pipeline" \
  --body "Adds crash-risk-ingestion/ (A1, unchanged), a combined requirements.txt, and up
dates prolog_interface.py: always retrains and uses models/best_model.joblib, predicts wi
th the model's trained feature columns, and adds --from-a1 to run A1's data_ingestion.py
and check its output against the data contract. Run: cd hybrid_ai_starter && python3 prol
```

```
og_interface.py --from-a1 --all. Needs train_models.py to save models/best_model.joblib."
\
  --reviewer Spyder-9999
```

It prints the PR's link when it's done. You can edit the description on GitHub afterwards, for example to paste in the fuller version. - **If it is listed**, note its state. If it shows **CLOSED**, it was closed by mistake: open it from that list and click **Reopen pull request**. If it shows **OPEN**, the Pull requests tab is probably filtered. Clear the search box, or go straight to github.com/Spyder-9999/HybridAI/pulls.

If `--reviewer` gives an error, run it again without that line and add Patrick as a reviewer on GitHub instead.

Prompt 36 (2026-11-03 06:18 UTC)

[image]

Response 36

Your PR is there: **#2** on the `integrate-a1-a2` branch. Going by the colours, it has probably already been **merged**. `gh` colours each PR number by state:

PR	Branch	Colour	Likely state
#4	update-readme	green	open
#3	update-train-predict	purple	merged (Kieren's `train_models.py` / `predict_site.py`)
#2	integrate-a1-a2	purple	**merged** (yours)
#1	merge-part1-part2	red	closed (Kieren's scrapped PR)

PR #2 was created 19 minutes ago, but you pushed the fix commit (`d766f1c`) more recently. **If #2 was merged before that push, the fix is only on your branch, not in `main`**, and `main` would have the version that fails with Kieren's model (the `resp` error). Check:

```
gh pr view 2 --json state,mergedAt
git fetch origin
git branch -r --contains d766f1c
```

- **If the last command lists `origin/main`**, the fix is in `main` and you're done.
- **If it only lists `origin/integrate-a1-a2`**, the fix missed the merge. Open a small follow-up PR with it:

```
gh pr create --base main --head integrate-a1-a2 \
  --title "prolog_interface: predict with the model's trained features" \
  --body "Follow-up to #2. Kieren's new train_models.py selects features from the data co
ntract (adds resp), so prediction now uses the saved model's own feature columns, and --f
rom-a1 checks A1 output against the data contract. Without this, the pipeline fails with
'columns are missing: {resp}'." \
  --reviewer Spyder-9999
```

If GitHub says there's nothing to compare, the fix is already in `main`.

Paste the output of those three commands and I'll confirm where things stand. After that, the last step is the live run on `main`.

Prompt 37 (2026-11-03 06:19 UTC)

[image]

Response 37

PR #5 has been merged, so the fix is in `main`. The badge at the top still says "Open" only because the page hasn't refreshed; the box at the bottom ("Pull request successfully merged and closed") is the current state.

`main` now has everything:

- **#2:** A1 folder, combined `requirements.txt`, and your interface with `--from-a1`
- **#3:** Kieren's four-model `train_models.py`, which saves `best_model.joblib`
- **#5:** prediction using the model's trained features, plus the data contract check

You can click **Delete branch**. It's safe, since everything on `integrate-a1-a2` is now in `main`.

Last step: one full live run on your Mac

```
cd ~/Documents/HybridAI
git checkout main
git pull
pip install -r requirements.txt
brew install swi-prolog
cd hybrid_ai_starter
python3 prolog_interface.py --from-a1 --all
```

Skip the `brew` line if SWI-Prolog is already installed (`swipl --version` will tell you).

What a good run looks like:

1. A1's log lines show the fetch, validation and transform, then the "Ingestion phase complete: N sites handed to Artefact 2" line.
2. Kieren's comparison table shows four models, followed by a "Best model: …" line.
3. A line reading "Training phase complete: selected model is RandomForestClassifier" or similar.
4. The Prolog step, then a summary with the number of sites in each action.

This is the first time the live A1 download runs as part of the whole pipeline, so if anything errors, paste the last 20 or so lines here.

Prompt 38 (2026-11-03 06:21 UTC)

how would someone in my group who isnt on a mac run it

Response 38

The steps are the same on Windows. Only the commands differ. Your A1 pipeline already ran on Windows when you built it, and the interface already looks for SWI-Prolog in its default Windows install folder.

Windows: one-time setup

1. **Python 3.11 or newer** from python.org. Tick **"Add python.exe to PATH"** in the installer. (pandas 3 in the requirements needs 3.11 or later.)

2. **SWI-Prolog** from swi-prolog.org (Download → Stable release → Windows 64-bit installer). Choose **"Add swipl to the system PATH"** in the installer. If they miss it, the interface still finds it in `C:\Program Files\swipl\bin\swipl.exe`.
3. **Get the repo**, either with Git:

```
gh repo clone Spyder-9999/HybridAI
```

or **Code → Download ZIP** on GitHub, then unzip. The repo is private, so they need to be a collaborator and signed in either way.

4. **Create the environment and install packages**, in PowerShell:

```
cd HybridAI
py -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

If PowerShell refuses to run `Activate.ps1`, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once, or use Command Prompt with `.venv\Scripts\activate.bat`.

Windows: running it

```
cd HybridAI\hybrid_ai_starter
python prolog_interface.py --from-a1 --all
```

The only differences from the Mac version are `python` instead of `python3`, and backslashes in paths. The flags are identical.

Linux works the same way as the Mac, except SWI-Prolog comes from the package manager (`sudo apt install swi-prolog` on Ubuntu or Debian).

If something goes wrong on Windows

- **"swipl not found":** close and reopen the terminal after installing SWI-Prolog, so the updated PATH is picked up.
- **pip fails on geopandas, pyproj or shapely:** usually a Python version without ready-made packages. Use Python 3.11–3.13.
- **Every new terminal** needs the environment activated again (`.venv\Scripts\Activate.ps1`) before running.

Worth adding to the repo README so anyone in the group can follow it without asking. Do you want me to write that section?

Prompt 39 (2026-11-03 09:44 UTC)

what are the subheadings for the report (artefact 2)

Response 39

[Tool action: Bash — Locate the A2 brief files]

[Tool action: Bash — Extract text of A2 brief and rubric]

These are the sections the A2 brief requires, in order, with a suggested word budget for the ~2,000 words (±10%, references included):

Title page Topic code, assessment number, name/FAN, campus, date and report title. Not counted in the words.

1. Capstone and Team Integration Statement (max 200 words) Your role (integration lead: `prolog_interface.py`), the overall system the team built, and how your module ties the team's work together.

2. Introduction (~250 words) - 2.1 Problem Statement and Objectives: recap from A1 - 2.2 Hybrid System Rationale: why ML plus symbolic, and what each part does

3. Dataset and Modelling (~300 words) - 3.1 Modelling: preprocessing, feature selection and engineering (the leakage exclusions), data types, and how the data was assessed

4. Hybrid System Architecture and Implementation (~550 words) - 4.1 Design: architecture diagram of the automated pipeline, showing: - the ML training phase - the inference phase: load model → predict → pass to Prolog - 4.2 Automated Training Pipeline and Algorithm Selection: why two or more algorithms (LR, DT, RF, XGBoost) - 4.3 Symbolic AI System Design: facts and rules in the knowledge base, and how they interpret the ML output into a recommendation - 4.4 Tools, Libraries and Methods: justification (scikit-learn, joblib, SWI-Prolog via subprocess, and so on)

5. Evaluation and Analysis (~550 words) - 5.1 ML Model Performance and Model Selection: metrics, comparison, why Random Forest - 5.2 Hybrid System Examples: input data → ML prediction → final output - 5.3 Overall Hybrid Performance: the value the symbolic layer adds, and limitations - 5.4 Consequences of Failure: safety and societal implications, and how the symbolic layer reduces those risks compared with ML alone

6. Conclusion (~150 words) Summary, key findings, future improvements.

References (IEEE, including the dataset and the AI tools)

Appendix: AI Usage Declaration The declaration statement, the model used with dates, and the full prompts and outputs. This needs its own heading with exactly that title, and it isn't counted in the words.

Formatting rules to keep in mind

- Single spacing, 12pt Times New Roman, 1.27cm side margins.
- FAN right-aligned in the header and page numbers in the footer on every page except the title page.
- Every figure and table numbered, with a caption.

The brief also says the AI declaration should appear at the start of the submission, so put a short statement near the front as well as the full appendix.

The rubric marks the report on how clearly the architecture is justified and how much value the symbolic layer is shown to add. Sections 4 and 5 are where the marks are, which is why they get the most words.

Prompt 40 (2026-11-03 10:15 UTC)

is the report in regards to the artefact 2 part specifically or the entire project (artefact 1 and 2)

Response 40

The report is about **Artefact 2**. Artefact 1 only appears as background and as the source of the data.

The brief brings A1 in at one point: section 2.1 asks you to "briefly recap the case study, problem statement, and overall project objectives from artefact 1." Everything else (modelling, architecture, evaluation, failure analysis) is about the hybrid ML + Prolog system.

Where A1 fits in your report

- **2.1 Problem Statement and Objectives:** a short recap of the problem and objectives. A few sentences, not a re-run of the A1 report.

- **3.1 Modelling:** one or two sentences saying the dataset comes from the A1 ingestion pipeline (validated, with zero-crash sites included), with a pointer to A1 and the data contract. The section itself covers A2's preprocessing and feature selection.
- **4.1 Design, and your Team Integration Statement:** because your module now runs A1 with `--from-a1`, the ingestion step legitimately appears as the first box in your architecture diagram. That's part of your A2 integration work, so describe how it's connected (the A1 output check and the handoff), not how A1 itself works.

What to leave out: detailed A1 content such as the CKAN fetch, SHA-256 hashing, the zip guards, the pandera rules and the reprojection. That was marked in A1, and repeating it uses up words that the A2 sections need. If something matters for the explanation, a single reference to your A1 report covers it.

Prompt 41 (2026-11-03 11:36 UTC)

so the system is 3 parts? machine learning/prediction then symbolic AI system. What is the first part called (artefact 1)

Response 41

The A2 brief formally defines the hybrid system as **two subsystems**: machine learning and symbolic AI. Artefact 1 is the **data ingestion pipeline** (the data layer). It sits before both and feeds them, but the brief doesn't count it as one of the hybrid subsystems.

The whole system as built:

Stage	Name	What it does	Where
1	**Data Ingestion Pipeline** (A1)	fetch → validate → transform → load; produces `intersection_sites.csv` and the data contract	`crash-risk-ingestion/`
2	**Machine Learning Subsystem**	trains and compares models, saves the best, predicts P(any_fsi)	`train_models.py`, `predict_site.py`
3	**Symbolic AI Subsystem**	Prolog rules combine the ML output with site facts to give an action and a reason	`crash_risk_rules.pl`
—	**Integration layer** (your module)	runs stages 1–3 in order and passes data between them	`prolog_interface.py`

In your report, "Data Ingestion Pipeline" (or "data ingestion layer") is the clearest name for A1. It matches your A1 report, and it signals that it's upstream data engineering, not one of the two hybrid subsystems the brief asks you to evaluate. Your own module is best described as the **integration layer** or **orchestration layer**: the component that turns the three stages into one automated pipeline. That makes your role easy to identify in the Team Integration Statement.

Prompt 42 (2026-11-03 11:38 UTC)

so what does the load part of data ingestion do again? what relevance is the intersection_sites.csv?

Response 42

[Tool action: Bash — Read A1 load stage and contract tables]

Load is A1's final stage (`ingestion/contract.py`, `write_outputs`). After fetch, validate and transform have produced clean tables, Load:

1. **Converts each table to the output coordinate system.**

2. **Writes each table in two formats** to `data/processed/`:
   - **GeoParquet** (`.parquet`): keeps the geometry and exact data types, for Python code that does spatial work.
   - **CSV** (`.csv`): flattens the geometry to a `lon`/`lat` point, for plain tools and anything that doesn't need spatial data.
3. **Generates the data contract** (`data_contract.json`). For every table it records the grain (what one row is), the primary key, the row count, and each column's type, null count, unit and meaning. It's built from the actual output, so it can't drift from the data.

`data_ingestion.py` then writes `run_report.json` (row counts, rejected rows, timings, source hashes).

A1 produces three tables:

Table	One row per	Rows	Used by A2?
`crash_locations`	crash location	43,522	No
`intersection_sites`	signalised intersection, **including zero-crash sites**	677	**Yes**
`segment_sites`	state-road segment	2,685	No

Why `intersection_sites.csv` matters: it's the **handoff file between A1 and A2**, the only A1 output the hybrid system uses. Each row is one intersection, with:

- **site features**: traffic volume (`entering_aadt`), heavy-vehicle share, crossings nearby, distance to the CBD, volume age, responsible authority (`resp`), and so on
- **crash outcomes**, including the target **`any_fsi`** (1 if the site had at least one fatal or serious-injury crash)

A2 uses it in three places:

- **Training:** `train_models.py` uses the features to predict `any_fsi`, leaving out the crash-outcome columns to avoid leakage.
- **Facts:** `prepare_site.py` turns a site's row into Prolog facts (traffic volume, crossings, volume stale or current).
- **Inference:** the best model scores each row. Your interface also checks that this file has the right columns before passing it on.

Including zero-crash sites is what makes it usable for modelling. Without them every site would have crashes, and the model couldn't learn what a lower-risk intersection looks like.

One detail to be aware of: Kieren's `train_models.py` reads the contract from `crash-risk-ingestion/docs/data_contract.json`, the copy saved in the repo, not the fresh one Load writes to `data/processed/` on each run. They match as long as A1's output columns don't change. If a new download returns a different number of rows, his script just prints a row-count warning and carries on.

Prompt 43 (2026-11-03 11:41 UTC)

what did FSI stand for again

Response 43

FSI stands for **Fatal and Serious Injury**. A crash counts as FSI if anyone was killed or seriously injured.

In your data:

- **`fsi_crashes`** is the number of fatal crashes plus serious-injury crashes at a site (`cse_fat + cse_si`).

- **`any_fsi`** is 1 if the site had at least one FSI crash in 2020–2024, and 0 otherwise. That's the target the models predict.

Prompt 44 (2026-11-03 11:54 UTC)

can you give me a small bullet point section of what my prolog_interface was responsible for prior to the merge

Response 44

Before the A1 merge, `prolog_interface.py` was responsible for:

- **Running training automatically:** it ran `train_models.py` on every run, deleting any old `best_model.joblib` first, so the model was always freshly selected.
- **Loading the selected model:** it checked that `best_model.joblib` had been created, then loaded it for prediction.
- **Preparing site data:** it used `prepare_site.py` to pull a site's row from `intersection_sites.csv` and write it as Prolog facts.
- **Running ML inference:** it used the best model to predict P(any_fsi), then wrote the classification and probability as Prolog facts.
- **Running the Prolog reasoning:** it launched SWI-Prolog with `crash_risk_rules.pl` and both sets of facts. It declared the fact predicates dynamic, so sites with missing data didn't crash the run.
- **Reading Prolog's conclusions:** it parsed each site's action, reason, data issues and contextual reasons from Prolog's output.
- **Saving and reporting results:** single site to `hybrid_result.json`; all sites (`--all`) to `hybrid_results_all.csv` and `hybrid_summary.json`, plus a run log.
- **Handling errors:** it stopped with a clear message if training, prediction or Prolog failed, rather than producing partial output.

The merge added one responsibility on top: **running A1 first** (`--from-a1`), which checks A1's output against the data contract and hands it to A2.

Prompt 45 (2026-11-04 03:05 UTC)

in the development of artefact 1, what is the systems objective

Response 45

[Tool action: Bash — Read A1 README for stated objectives]

[Tool action: memory_read — /areas/crash-risk-project.md]

The Artefact 1 system's objective was to **build an automated, secure and repeatable data ingestion pipeline**. It turns five open South Australian datasets from data.sa.gov.au into validated, model-ready tables and a data contract for the team's machine learning and reasoning modules.

That breaks down into:

- **Acquire:** fetch the current datasets automatically through the CKAN API over HTTPS. Download only what changed, record a SHA-256 hash of every file, and fall back to cached or manual copies if the network fails.
- **Validate:** check types, ranges and domain rules with Pandera. Quarantine failing rows with the reason they failed, rather than silently dropping them.
- **Transform:** reproject everything to one coordinate system, fill gaps, and match crashes and traffic volumes to road sites by distance. Include sites with **zero crashes**, so the data isn't biased towards dangerous sites.

- **Load:** publish three tables (`crash_locations`, `intersection_sites`, `segment_sites`), plus a data contract describing every column, and a run report for traceability.

That served the overall project objective: **predicting crash risk and severity at South Australian road sites and recommending explainable safety treatments**. A1 supplied the trustworthy data, and A2 does the prediction and reasoning.

For section 2.1, a two-sentence version could be:

> Artefact 1 delivered an automated ingestion pipeline that fetches five South Australian open datasets, validates them against explicit schemas, and fuses them into model-ready site tables, including zero-crash sites, described by a machine-readable data contract. This provided a reliable, traceable foundation for the hybrid prediction and reasoning system built in Artefact 2.

This comes from the A1 code and README. Your A1 report itself isn't in my workspace, so compare the wording with the problem statement and objectives you wrote there, and keep the two consistent.

Prompt 46 (2026-11-04 03:06 UTC)

from the development*

Response 46

If you mean the objective of the whole system as defined during Artefact 1 (the one section 2.1 asks you to recap), it was this:

Overall system objective, set in Artefact 1: build a hybrid AI system that **predicts crash risk and severity at South Australian road sites** (signalised intersections and state road segments) and **reasons over those predictions to recommend explainable safety treatments**, so road authorities can decide where to focus limited safety resources.

The problem behind it:

- Crash data, traffic volumes and road-feature data are spread across separate public datasets, in different formats and coordinate systems.
- Raw crash counts mislead without exposure. A busy intersection naturally has more crashes, so volume has to be taken into account.
- A prediction on its own doesn't tell a decision-maker what to do or why. Recommendations have to be explainable to be trusted and acted on.

In A1, development covered the first step towards that objective: a validated, traceable data foundation, with ingestion producing the site tables and the data contract.

Wording to make consistent in your A2 report: A2 delivered a narrower version of that objective. It covers **signalised intersections only**, and predicts **`any_fsi`** (whether a site has had a fatal or serious-injury crash), not crash frequency. Say so explicitly in 2.1 or 3.1. A marker will compare the objective you restate with what you evaluate, and naming the narrower scope yourself reads as deliberate rather than as falling short.

As before, this is based on the A1 code and README, not your A1 report, so match the wording to what you wrote there.